Lecture 9: Core Machine Learning, part 3

LSE ME314: Introduction to Data Science and Machine Learning (https://github.com/me314-lse)

2026-07-27

Ryan Hübert

Slides last updated: 24 July 2026

Administrative information

Final exam information has been posted to GitHub.

- ➜ https://github.com/me314-lse/study-guide

Thursday:

- ➜ Review session in seminar.
- ➜ Only questions on the course material.

Midterm marks:

- ➜ Come directly from SSO.
- ➜ Some feedback will be posted to Moodle.

Seeing patterns in messy data

Unsupervised learning

Recall: **unsupervised learning** is used in situations where:

1. All values for Y are missing.
2. You don't have a clear sense of what Y even is.

But, what does it allow you to do?

➜ Identify interesting patterns in the features (X variables).

➜ Use these patterns to come up with new concepts, new measures, etc.

Unsupervised learning

The big picture of unsupervised learning: reducing complexity.

What does this mean?

- ➜ A lot of data looks like a jumbled mess to the human eye.
- ➜ Datasets are "highly dimensional": lots of moving parts...
  - ➜ Many different observations.
  - ➜ Many different variables.
  - ➜ Complex relationships between variables and observations.
- ➜ Ideally want to organise it in ways that help us see new things.
- ➜ Most of us can't do it when looking at big datasets.
- ➜ Can we reduce this to something more manageable?

Some examples

- YouTuber currently offers a monthly subscription for £5.
- She recently gained a lot of subscribers.
- She suspects that some of her subscribers are more devoted than others.
- She has a bunch of data on each subscriber.
- She wants to segment subscribers into clusters based on their engagement patterns to:
  - Tailor content and perks for each group.
  - Introduce tiered pricing (e.g., £5 / £10 / £20).

Some examples

- University IT department monitors system activity: login times, IP addresses, access patterns, and data transfer volumes.
- Want to detect potential security breaches.
- But cyberthreats are evolving and they don't have an exact idea of what they should be looking for.
- They monitor logs for anomalies and unusual activities.

Some examples

- A researcher fields a survey asking respondents a series of questions about their policy views.
- The researcher wants to know whether there are any underlying worldviews that shape views.
- For example: are views affected by how liberal or conservative respondents are?
- But she doesn't know what these underlying worldviews are.
- She looks for patterns in survey responses that indicate worldviews.

Some background: the geometry of data

Data as points in a (vector) space

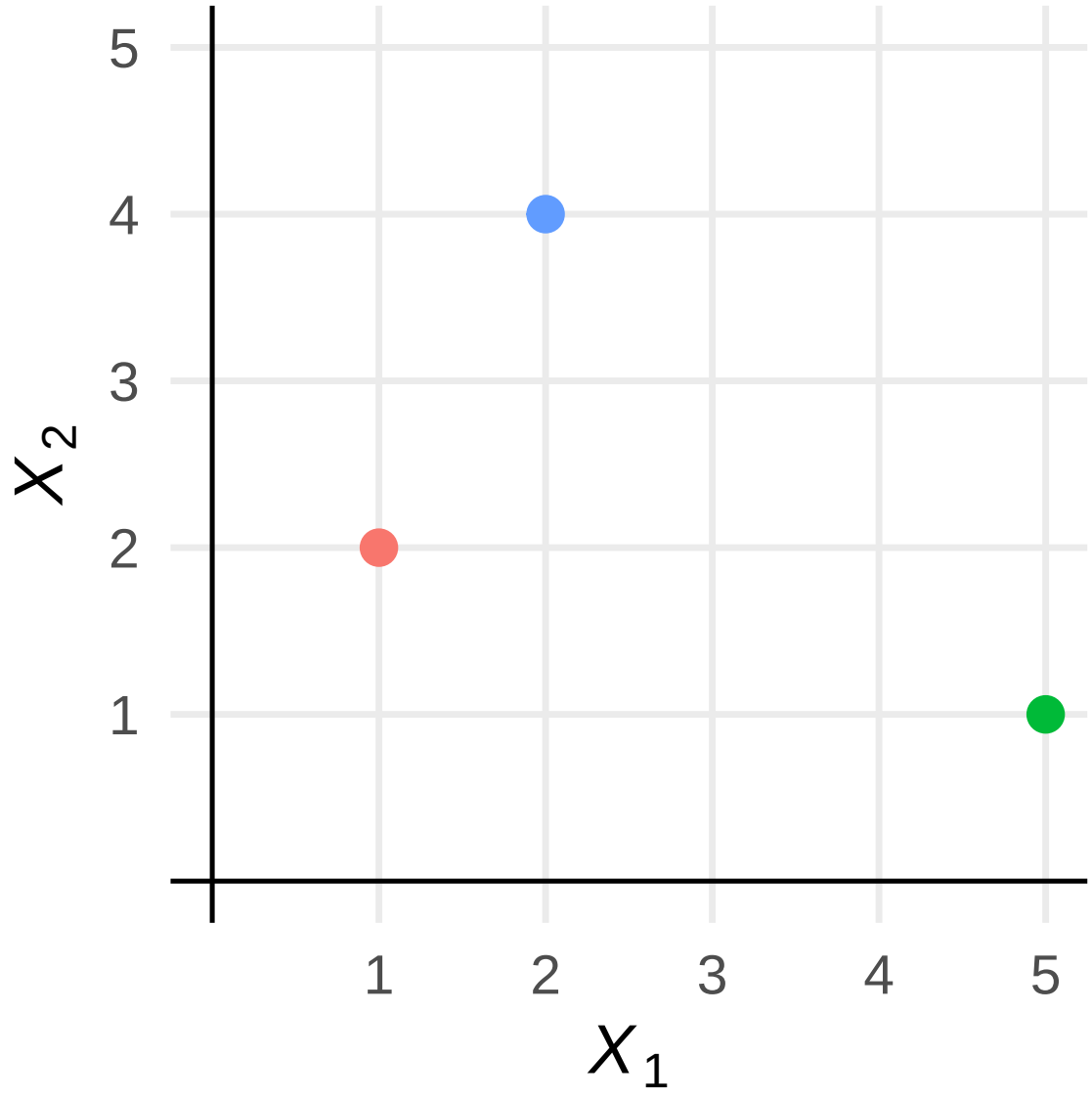
Consider a tiny dataset:

- Three observations ($N = 3$).
- Two features ($J = 2$) called X_1 and X_2.

```
# A tibble: 3 x 2
     X1    X2
  <dbl> <dbl>
1     1     2
2     5     1
3     2     4
```

We can visually depict this dataset using a diagram you already know and love: the scatterplot.

Data as points in a (vector) space



Data as points in a (vector) space

But what *is* a scatterplot at a deeper level?

- ➜ It visualises observations for two features of a dataset.
- ➜ Each feature X_1 and X_2 is depicted by an axis, corresponding to a **dimension**.
- ➜ The two dimensions create a **vector space**.
- ➜ Since we can only cleanly draw two dimensions on a flat surface (like a lecture slide), we only visualise two features at a time.
  - ➜ This 2 dimensional (vector) space is called the **Cartesian plane**.
- ➜ The observations $\mathbf{x}_i = (X_{1i}, X_{2i})$ fall into this vector space.
  - ➜ Each point can be represented as a **vector** in the vector space.

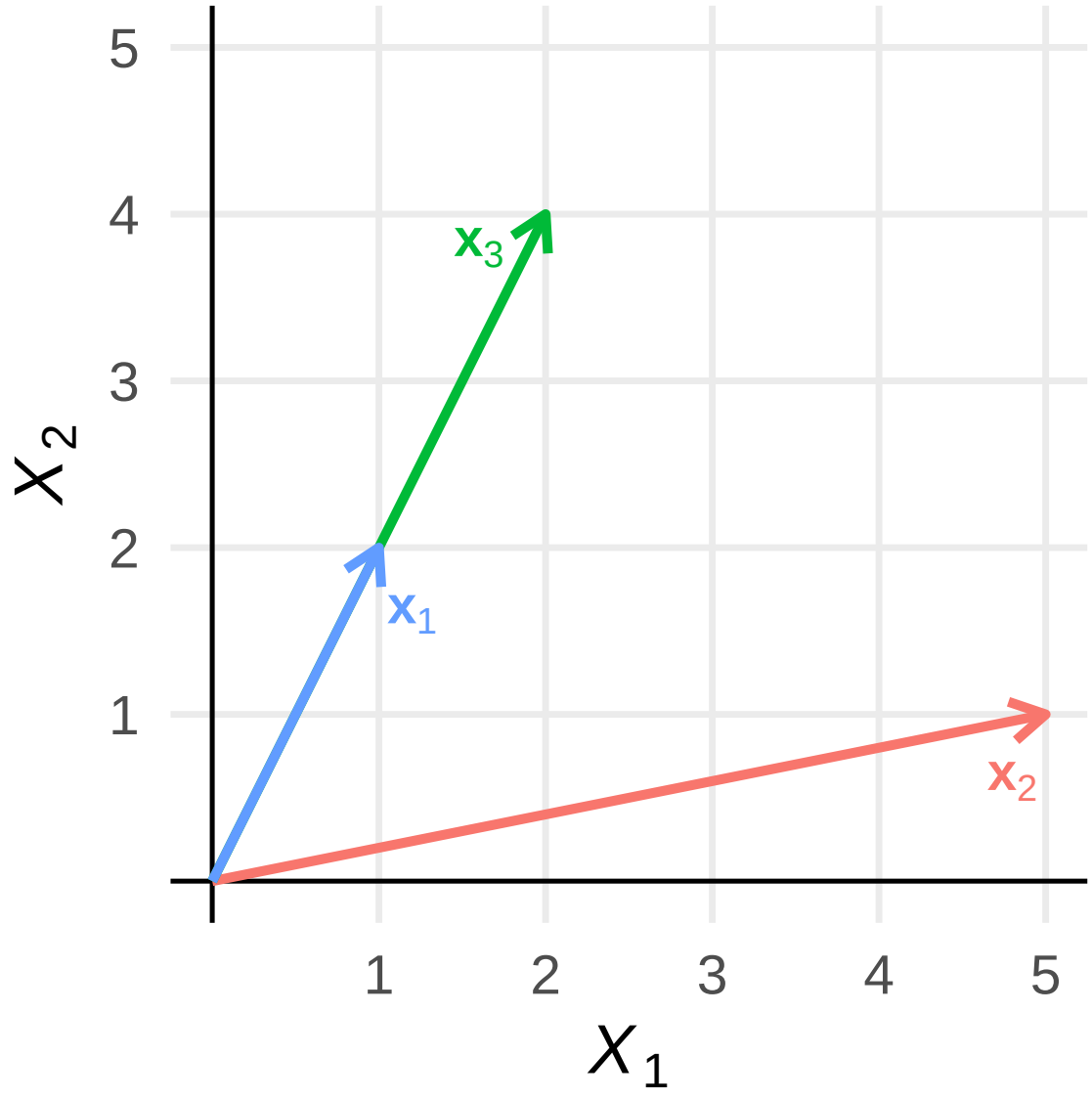
Data as points in a (vector) space

Please note our tricky math notation:

- X_1 is the name of the first feature (column/variable)
  - `R` doesn't allow subscripts, so variable is named `X1` (yikes!)
- $\mathbf{x}_1$ is a vector representing the first row: $\mathbf{x}_1 = (1, 2)$
- X_{1i} is the value of a specific observation i in column 1
  - Remember: I am a weirdo who likes to index the column before the row in this case.

We won't use it here, but we could use $\mathbf{X}_1$ if we wanted to refer to the values in the entire first column: $\mathbf{X}_1 = (1, 5, 2)$

Data as points in a (vector) space



Data as points in a (vector) space

Why think of data this way?

Useful for contexts where **similarity** is important.

- ➜ Vectors are *geometric* representations.
- ➜ They are contained in vector spaces where there are well-defined notions of **distance** and **direction**.
- ➜ The "closeness" of observations has mathematical meaning.

Data as points in a (vector) space

This isn't *really* new.

Every time you draw a scatter plot, you are:

1. Drawing a vector space (a Cartesian plane) when you draw the axes, and

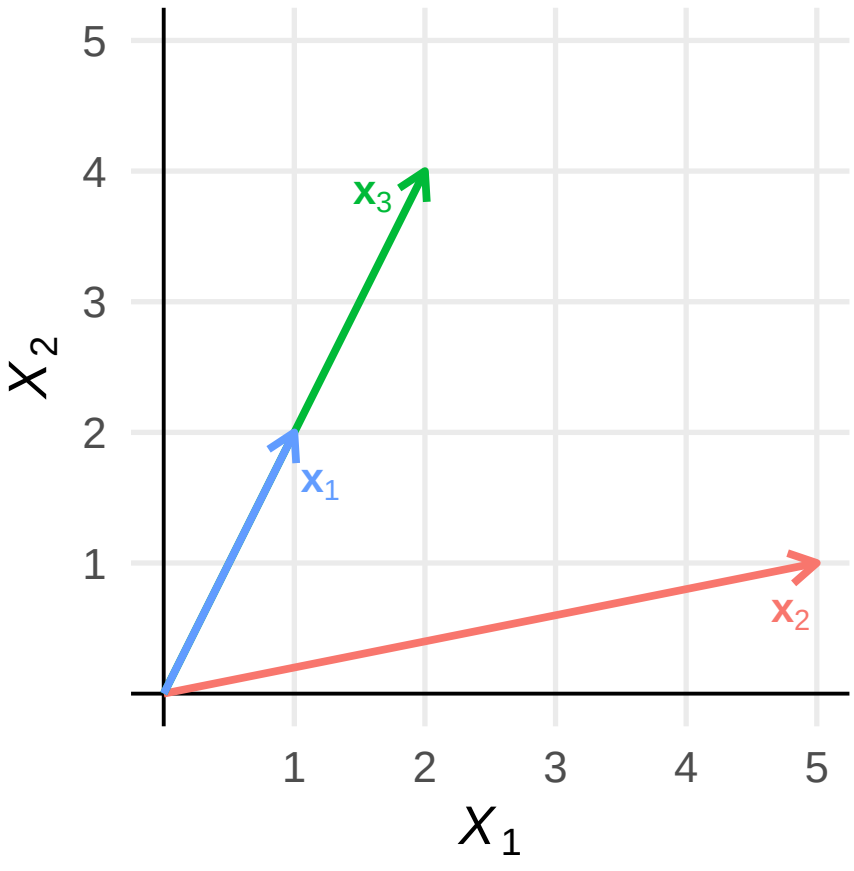
2. Drawing the end points of every vector in your dataset (every observation).

Even when you view a dataset as a table, you're looking at a **matrix**, which is the observations (vectors) stacked on top of each other.

The new thing here: drawing observations as arrows.

➜ Helps us think about directions.

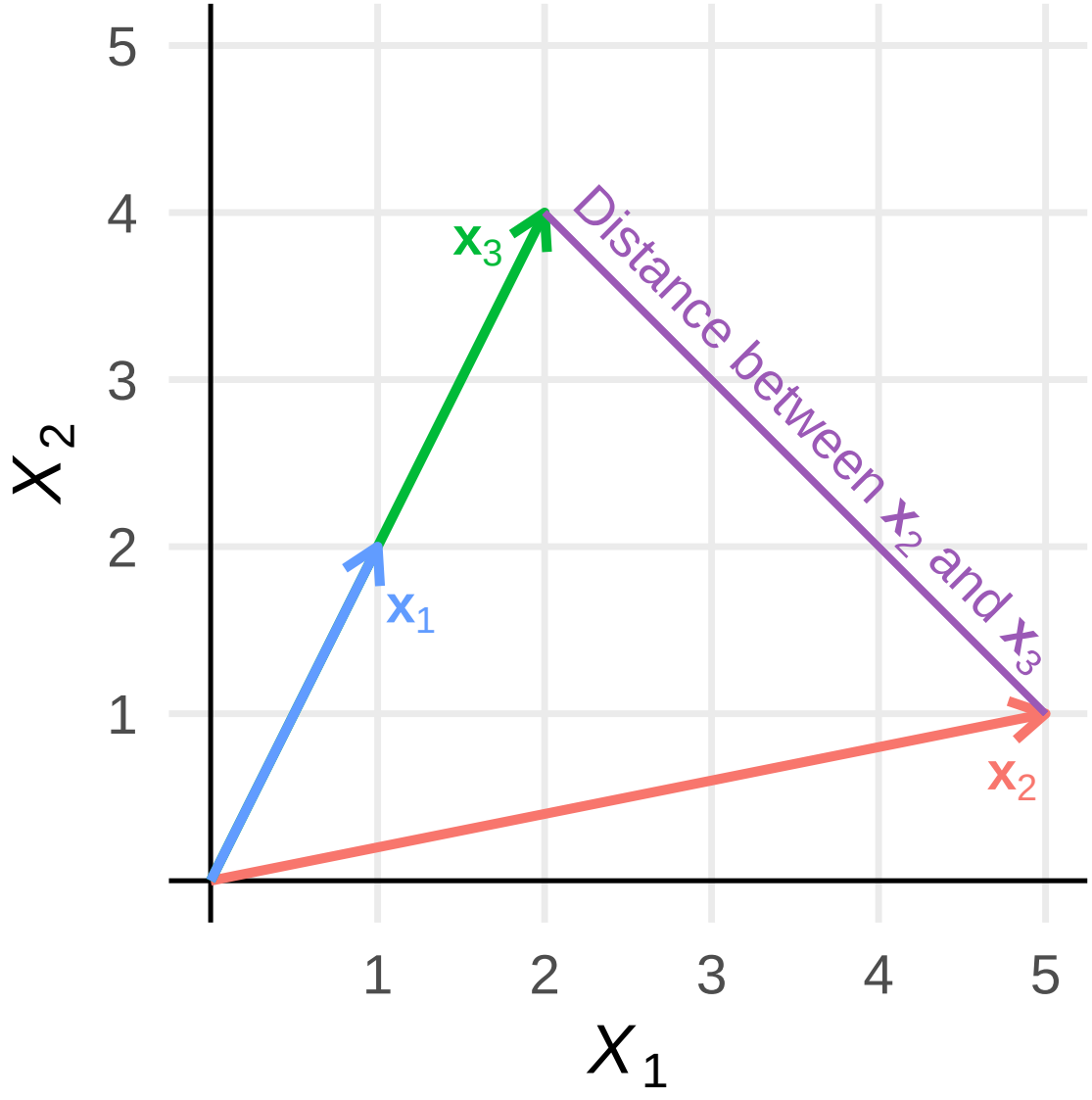
Data as points in a (vector) space



How do we *compare* these observations?

1. Distance: absolute separation between points
2. Similarity: directional closeness of vectors

Distance: absolute separation between points



Euclidean distance

The previous slide illustrated the **Euclidean distance** between observations $\mathbf{x}_2$ and $\mathbf{x}_3$.

➜ Euclidean distance is based on the Pythagorean theorem.

For two observations $\mathbf{x}_A$ and $\mathbf{x}_B$ with J features:

$$d_E(\mathbf{x}_A, \mathbf{x}_B) = \|\mathbf{x}_A - \mathbf{x}_B\| = \sqrt{\sum_j (X_{jA} - X_{jB})^2}$$

For two observations $\mathbf{x}_A$ and $\mathbf{x}_B$ with 2 features X_1 and X_2:

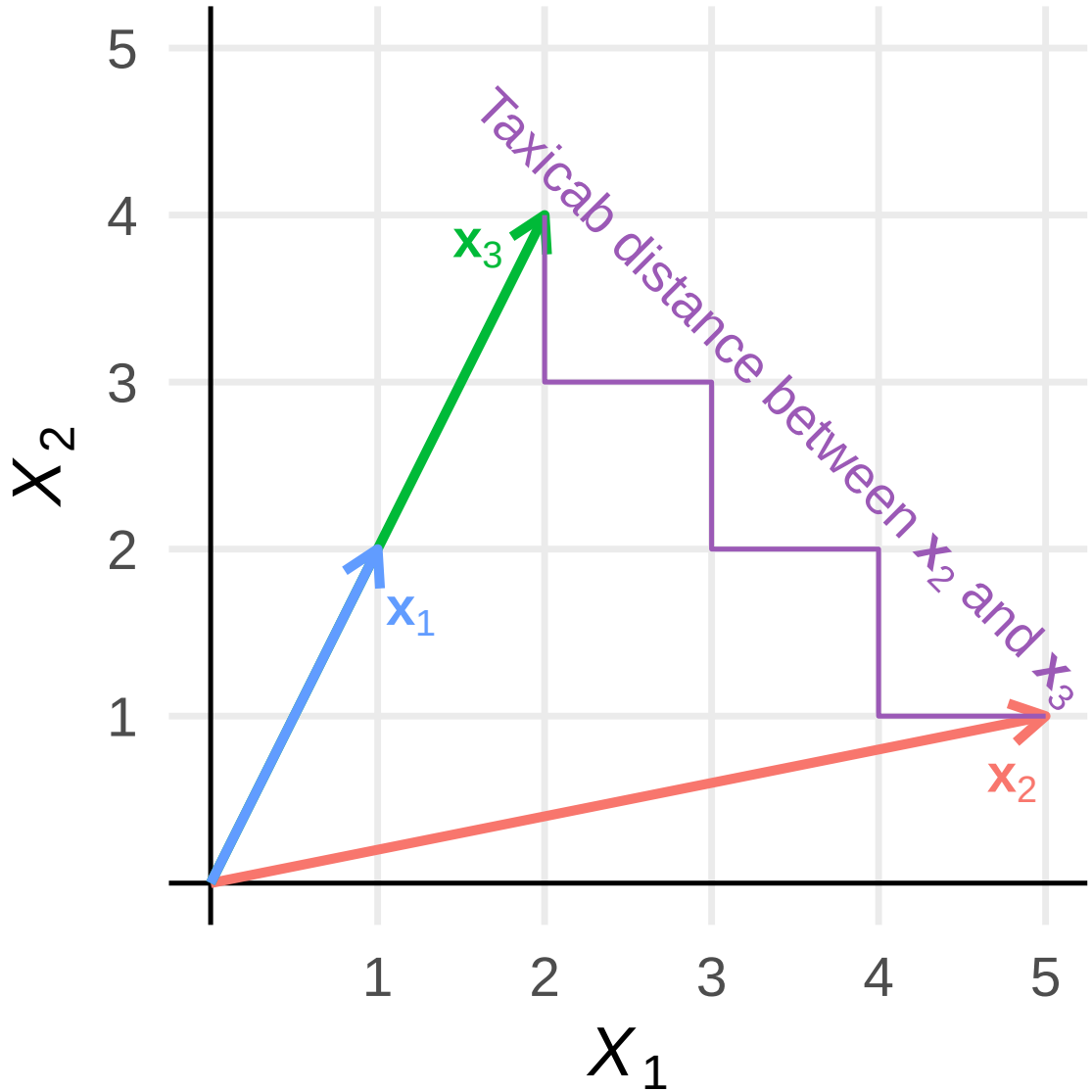
$$d_E(\mathbf{x}_A, \mathbf{x}_B) = \|\mathbf{x}_A - \mathbf{x}_B\| = \sqrt{(X_{1A} - X_{1B})^2 + (X_{2A} - X_{2B})^2}$$

So, for $\mathbf{x}_2 = (5, 1)$ and $\mathbf{x}_3 = (2, 4)$:

$$\|\mathbf{x}_2 - \mathbf{x}_3\| = \sqrt{(5-2)^2 + (1-4)^2} \approx 4.24$$

Other distance metrics

The **Manhattan distance** (or **taxicab distance**) is calculated assuming you can only measure in right angles.



Other distance metrics

It is calculated with this formula:

$$d_T(\mathbf{x}_A, \mathbf{x}_B) = \sum_j |X_{jA} - X_{jB}|$$

So, for $\mathbf{x}_2 = (5, 1)$ and $\mathbf{x}_3 = (2, 4)$:

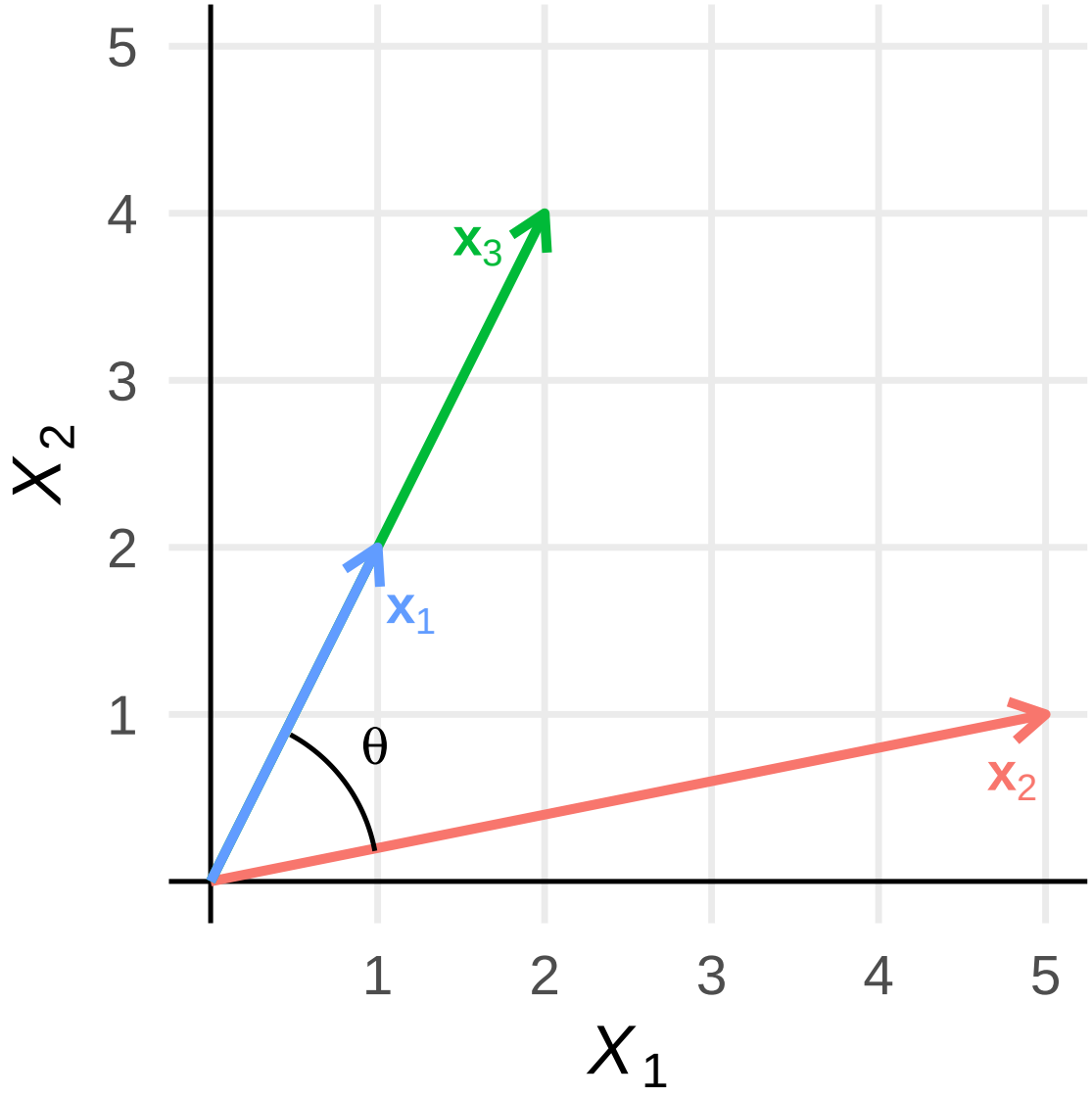
$$d_T(\mathbf{x}_2, \mathbf{x}_3) = |5 - 2| + |1 - 4| = 6$$

There are other distance metrics.

- ➜ Each has pros and cons, but this is a little bit too "in the weeds" for us.
- ➜ Euclidean distance is sensitive to vector magnitude: two observations with similar patterns but different scales will appear far apart.
- ➜ Cosine similarity avoids this by measuring only directional closeness.

Similarity: directional closeness

How *directionally* close are observations?



Cosine similarity

The **cosine similarity** between two observations $\mathbf{x}_A$ and $\mathbf{x}_B$ is:

$$\text{cosine similarity}(\mathbf{x}_A, \mathbf{x}_B) = \frac{\sum_j X_{jA} \cdot X_{jB}}{\sqrt{\left(\sum_j X_{jA}^2\right) \cdot \left(\sum_j X_{jB}^2\right)}}$$

Why is it called "cosine similarity"?

$$\text{cosine similarity}(\mathbf{x}_A, \mathbf{x}_B) = \frac{\sum_j X_{jA} \cdot X_{jB}}{\sqrt{\left(\sum_j X_{jA}^2\right) \cdot \left(\sum_j X_{jB}^2\right)}} = \cos\theta$$

Cosine similarity is based on the size of the angle between vectors—their *directional* closeness!

Cosine similarity

So, for $\mathbf{x}_2 = (5, 1)$ and $\mathbf{x}_3 = (2, 4)$:

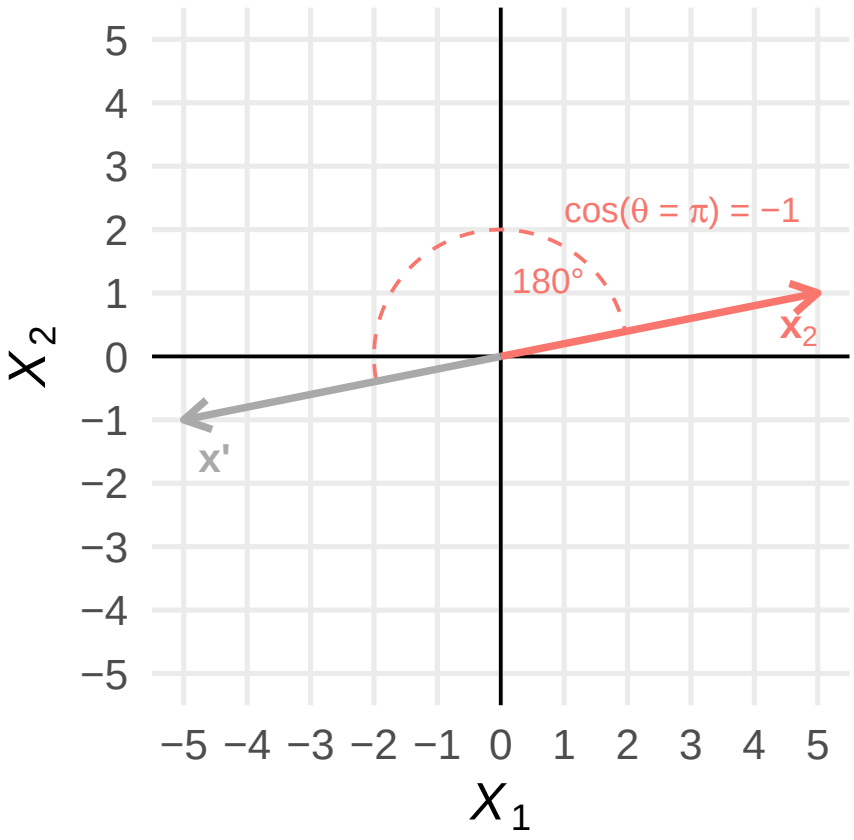
$$\text{cosine similarity}(\mathbf{x}_2, \mathbf{x}_3) = \frac{(5 \cdot 2) + (1 \cdot 4)}{\sqrt{(5^2 + 1^2) \cdot (2^2 + 4^2)}} \approx 0.614$$

And if we want to know the angle between these observations:

$$\cos^{-1}(0.614) \approx 0.91 \text{ radians} \approx 52°$$

There are other similarity metrics, but this is a very common one.

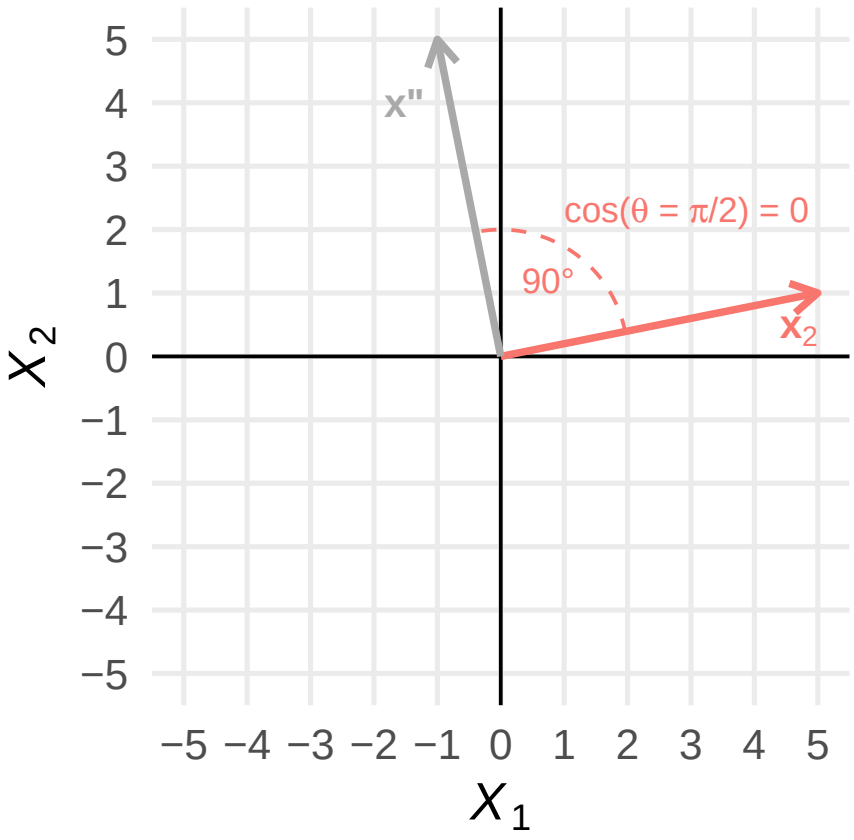
Cosine similarity



Metric ranges from -1 to 1:

- ➜ -1 is least similar ("opposite")
- ➜ 0 is **orthogonal** (90° angle, "unrelated")
- ➜ 1 is most similar (identical)

Cosine similarity



Metric ranges from -1 to 1:

- ➜ -1 is least similar (“opposite”)
- ➜ 0 is **orthogonal** (90° angle, “unrelated”)
- ➜ 1 is most similar (identical)

Principal components analysis

Example

Survey of voters' views on a variety of public policy questions.
Answers on scale of 1 (strongly oppose) to 5 (strongly support).

- Support publicly funded healthcare for all
- Support tuition-free higher education
- Support expanding access to abortion services
- Support taxes on carbon emissions
- Support looser regulations on gun ownership
- Support limits on immigration
- Support government funding for physical border security
- Support legal recognition of same-sex relationships
- Support access to gender-neutral public facilities
- Support progressive income taxation
- Support increased national defense spending
- Support expansion of foreign aid programs
- Support government regulation of social media platforms
- Support legal restrictions on hate speech
- Support replacing cash bail with risk-based detention
- Support raising the minimum wage
- Support increasing funding for law enforcement
- Support requiring government-issued ID to vote in elections

Example

	same_sex_marriage	progressive_taxation	increase_natl_def	tuition_free_edu
1	3	3	2	4
2	3	4	2	4
3	4	3	4	2
4	3	4	5	4
5	2	2	5	3
6	3	3	4	2

	access_abortion	restrict_hate_speech	raise_min_wage	govt_reg_social_media
1	2	3	5	1
2	2	2	4	2
3	4	5	2	5
4	3	3	4	3
5	2	3	3	4
6	2	3	3	4

	limit_immigration	border_security_funding
1	2	3
2	4	2
3	4	3
4	3	2
5	3	3
6	4	3

Principal components analysis

Maybe these voters' attitudes on various issues are related?

We have a dimension reduction goal:

- Start with features $X_1, X_2, ..., X_J$, and
- Transform them into a smaller number of features where:
  - All of the new features are uncorrelated.
  - The new features capture as much of the variance in the original features as possible.

Principal components analysis (PCA): a method to find linear combinations of the original features that are uncorrelated and that capture as much variance as possible.

Principal components analysis

For example, the first principal component:

$$\text{PC1}_i = \phi_{11} X_{1i} + \phi_{21} X_{2i} + \phi_{31} X_{3i} + \cdots + \phi_{J1} X_{Ji}$$

And the second:

$$\text{PC2}_i = \phi_{12} X_{1i} + \phi_{22} X_{2i} + \phi_{32} X_{3i} + \cdots + \phi_{J2} X_{Ji}$$

We will not get into details of the algorithm, but note that:

1. It produces PCs that are linear combinations of original features.
2. It generates PC1, PC2, etc. that are uncorrelated.
3. It produces as many principal components as original features—the human has to decide how many PCs to keep.

PCA estimates

After the algorithm has been run, we have:

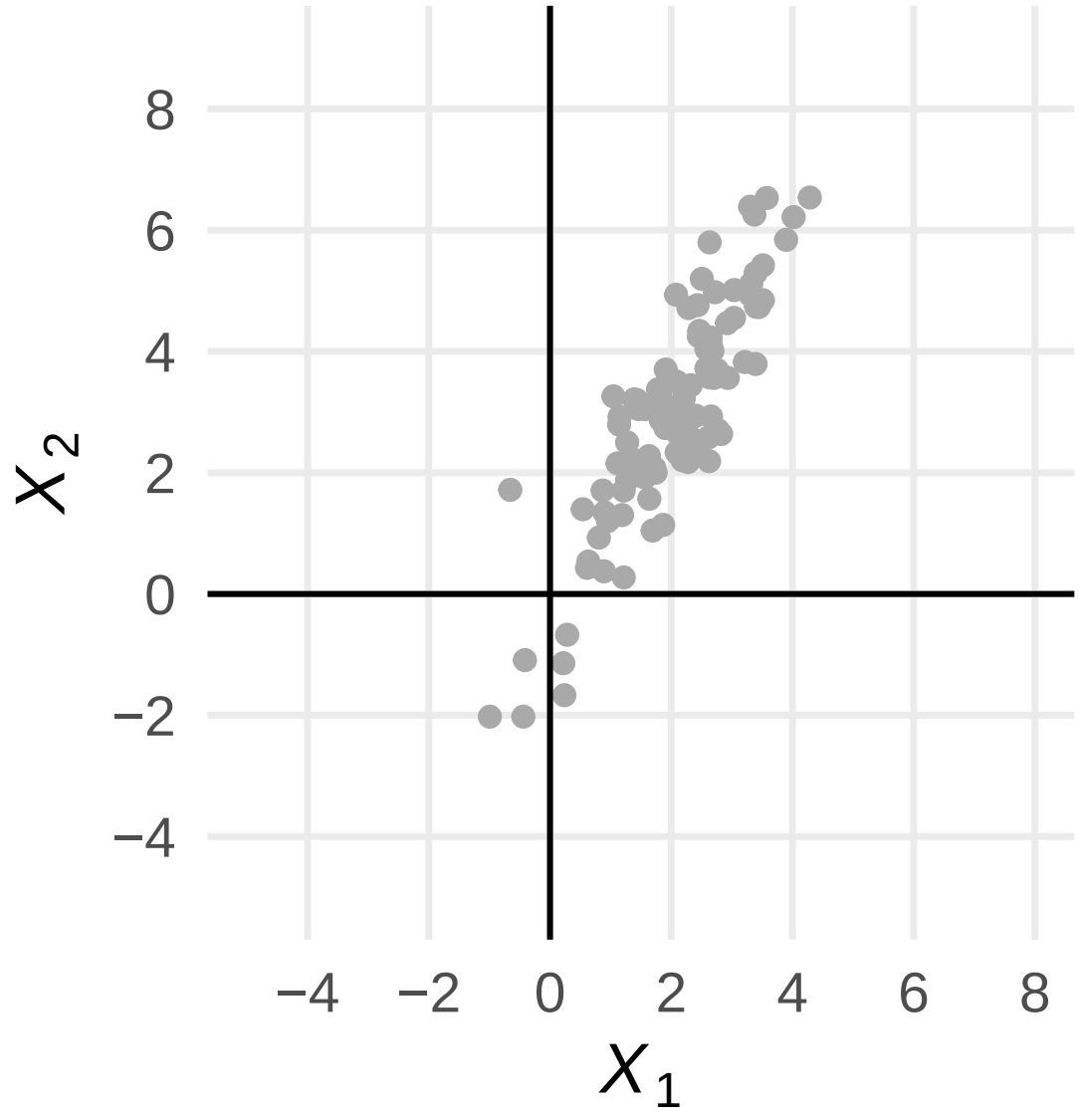
1. Estimated **loadings**: the estimated $\hat{\phi}_{jk}$ coefficients (one per feature j and PC k).

2. Estimated **principal component scores**: for each observation i, estimated $\widehat{\text{PC}}$s, calculated from the original data and the $\hat{\phi}$s.

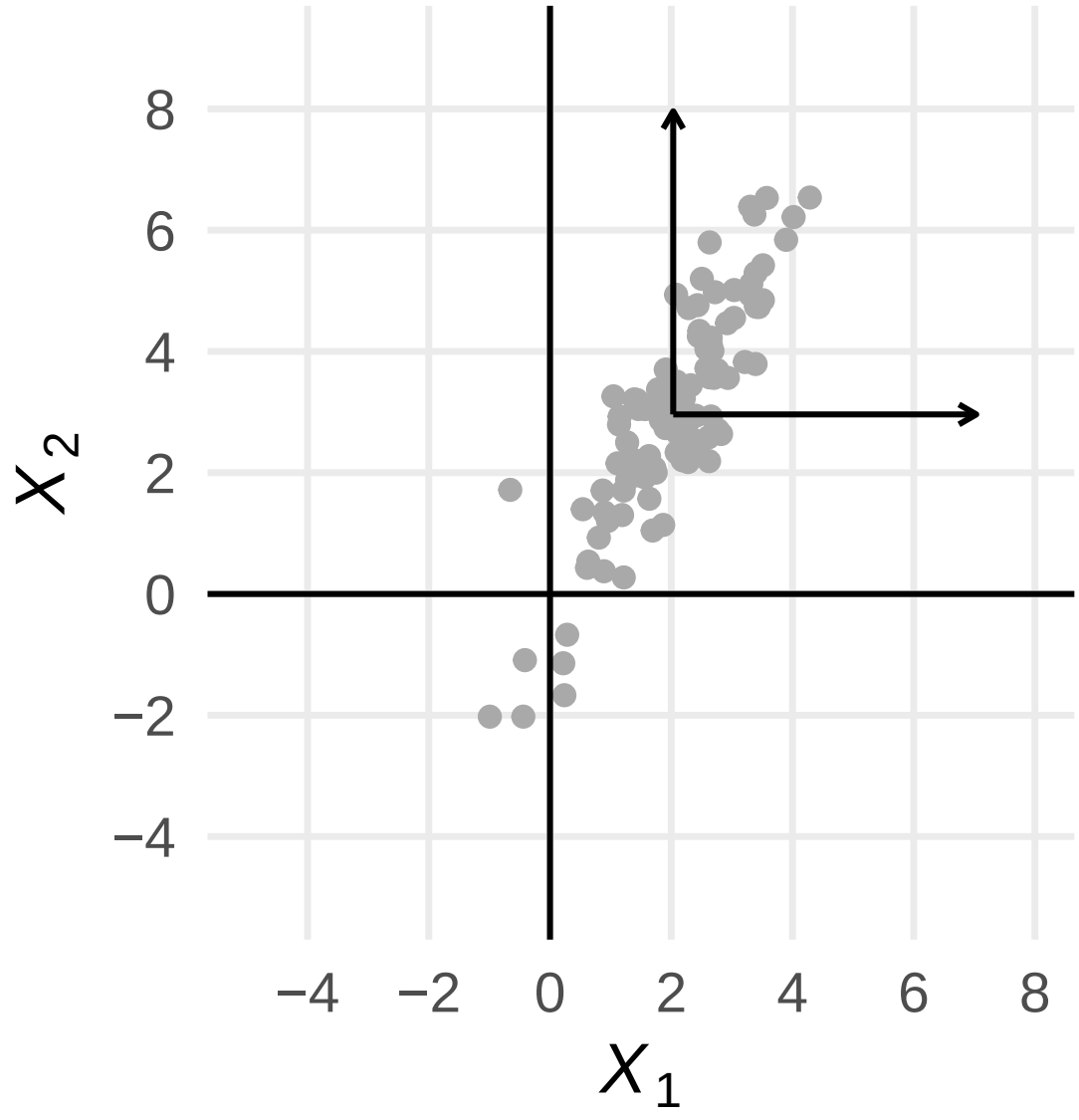
Note: typically you recentre and/or rescale before running algorithm:

- ➜ Recentre: subtract mean of variable from each observation.
- ➜ Rescale: divide observation (recentred or not) by standard deviation of variable.
- ➜ Calculate the $\widehat{\text{PC}}$s with the recentred/rescaled data.

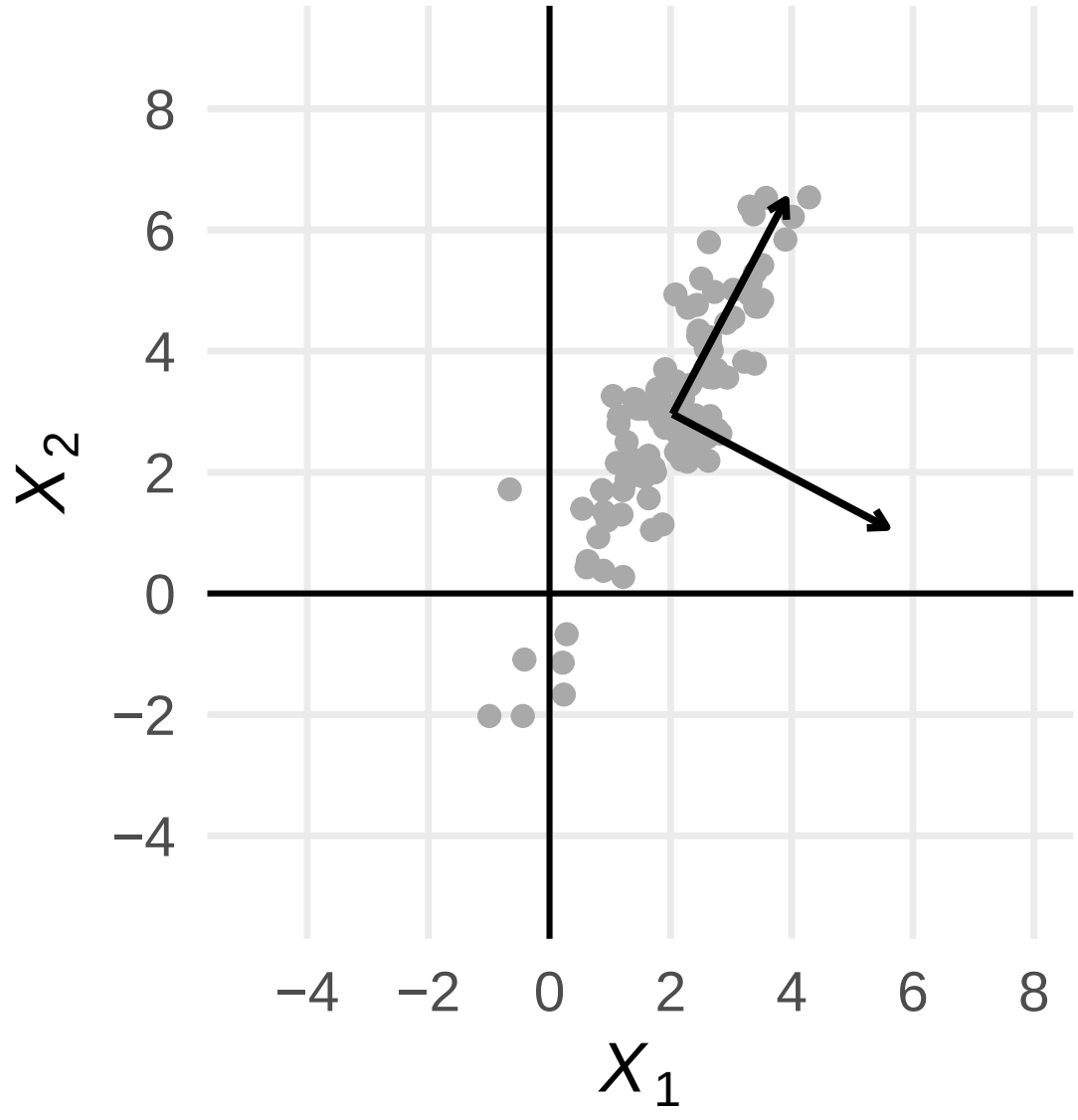
PCA example with 2 features



Recentring the data



Finding an optimal rotation



Finding an optimal rotation

After we run PCA, we get a **rotation matrix** that gives us the estimated values of the $\hat{\phi}$s:

```
          PC1        PC2
X1 0.4669761  0.8842700
X2 0.8842700 -0.4669761
```

So, we can write out the formulas to calculate the PC scores for each observation i:

$$\widehat{\text{PC1}}_i = 0.467\,(X_{1i} - \overline{X}_1) + 0.884\,(X_{2i} - \overline{X}_2)$$

$$\widehat{\text{PC2}}_i = 0.884\,(X_{1i} - \overline{X}_1) - 0.467\,(X_{2i} - \overline{X}_2)$$

For example, the first observation is $X_{11} = 3.37$ and $X_{21} = 6.26$ (and $\overline{X}_1 = 2.03$ and $\overline{X}_2 = 2.96$):

$$\widehat{\text{PC1}}_1 = 0.467\,(3.37 - 2.03) + 0.884\,(6.26 - 2.96) = 3.54$$

$$\widehat{\text{PC2}}_1 = 0.884\,(3.37 - 2.03) - 0.467\,(6.26 - 2.96) = -0.36$$

Choosing the number of "relevant" PCs

Recall: point is to *reduce* the number of dimensions.

But the algorithm gives us the same number of PCs as features.

So, we have to use our judgement to decide how many PCs to keep.

- ➜ This depends on your goal.
- ➜ Part science, part art.

The science of this: each PC statistically captures a certain amount of variance in the data.

- ➜ *By construction*, the first PC always captures most variation.

Make your decision partly based on eliminating PCs that don't capture much variance.

Choosing the number of "relevant" PCs

We can calculate how much variance each PC captures.

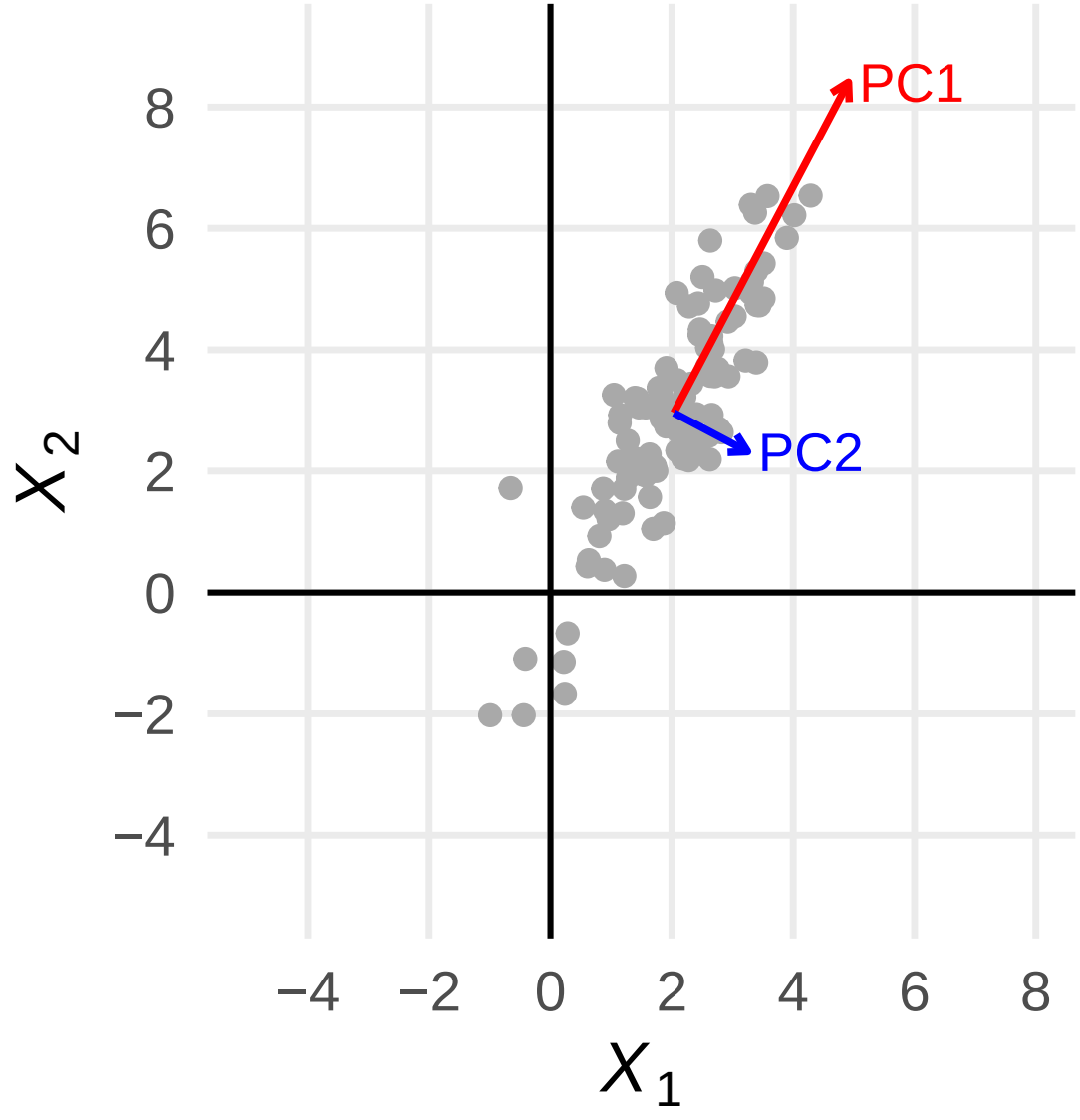
In our simple 2-feature example:

Proportion of variance explained:

```
    PC1      PC2
0.95263 0.04737
```

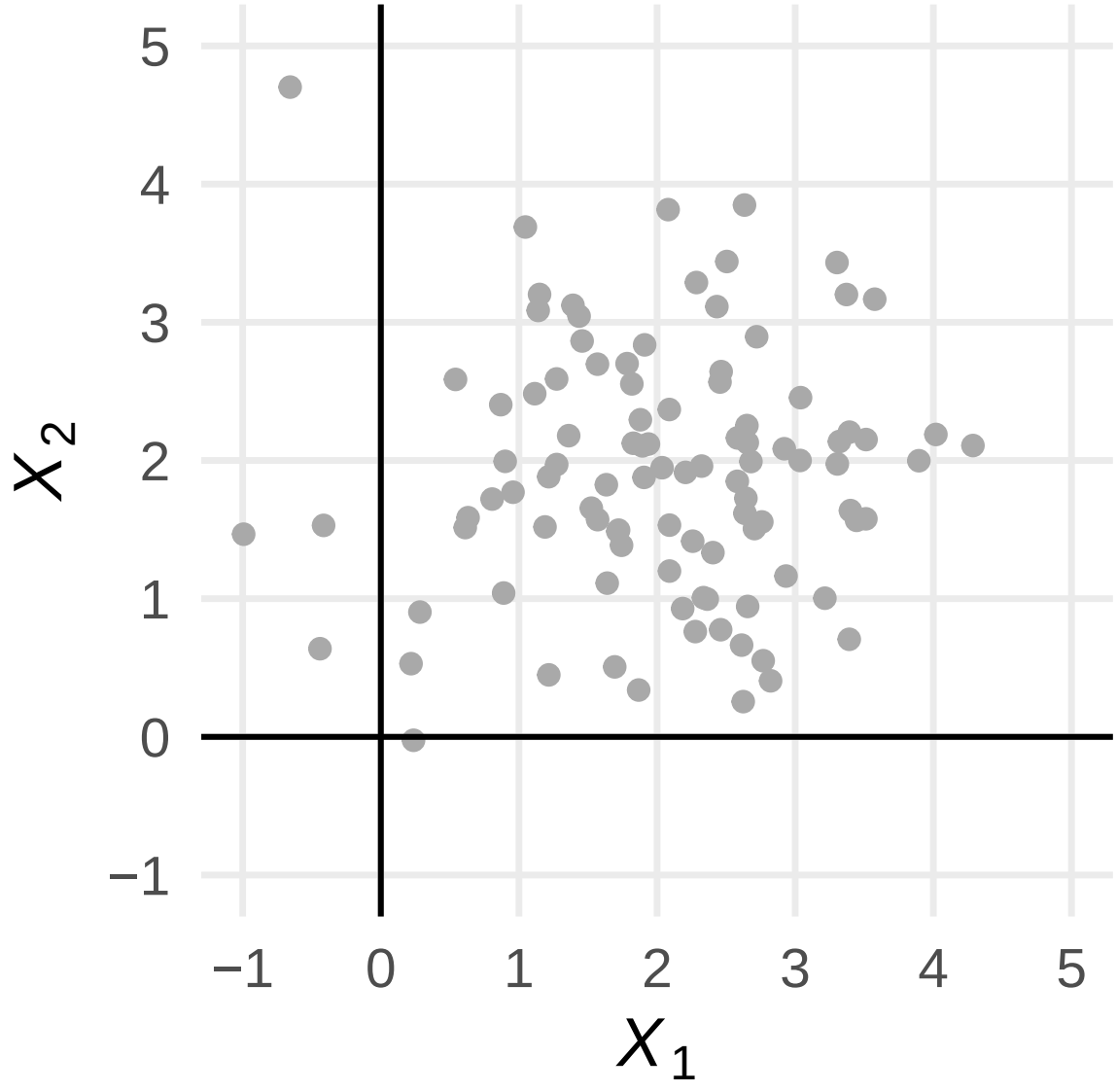
In this case, PC1 captures almost all of the variance in the data!

Choosing the number of "relevant" PCs



PCA doesn't always work well

PCA won't work as well for data without clear relationships in it:



PCA doesn't always work well

Visually, the variance in the data seems to be equally explained by both variables!

Can see this in proportion of variance explained:

```
    PC1     PC2
0.57185 0.42815
```

In this case, we probably don't want to do dimension reduction.

Doing PCA in R

Let's go back to the original example of the survey responses.

Run PCA in `R` using the function `prcomp` from the `stats` library.

➜ Note: `stats` is included in the default installation of `R`

```
pca_result <- prcomp(survey.df, scale. = TRUE)
print(pca_result$rotation)
```

We use `scale. = TRUE` to ensure that features with extreme polarisation in responses don't dominate PCs.

➜ E.g., X_{11} becomes $\frac{X_{11} - \overline{X}_1}{\widehat{\text{s.d.}}(X_1)}$.

Looking at the $\hat{\phi}$s

	PC1	PC2	PC3	PC4
same_sex_marriage	-0.52164048	-0.2185414369	0.090584402	0.04209690
progressive_taxation	-0.22828004	0.5274602974	-0.007637308	0.05338650
increase_natl_def	0.06622408	-0.0006090605	0.018527135	0.68859078
tuition_free_edu	-0.23309037	0.5319204543	-0.071496563	0.03453368
access_abortion	-0.52009136	-0.2310788229	0.099953331	0.00460798
restrict_hate_speech	-0.51786392	-0.2446320633	0.062260286	0.09332376
raise_min_wage	-0.25878022	0.5072353218	-0.068399801	-0.06328182
govt_reg_social_media	0.03878708	0.0199488279	-0.079173922	0.71138806
limit_immigration	0.06423964	0.0794934982	0.698761414	0.01460755
border_security_funding	0.07606740	0.1176368219	0.687895018	0.03172375

	PC5	PC6	PC7	PC8
same_sex_marriage	-0.071362459	0.30708496	0.259189047	-0.232422205
progressive_taxation	0.002746701	-0.02849772	0.712212324	0.077343490
increase_natl_def	0.711509108	-0.01781306	0.029900867	0.006403324
tuition_free_edu	-0.053122245	-0.26264887	-0.231203535	-0.583958530
access_abortion	-0.013527738	-0.33858128	0.089245120	0.486667383
restrict_hate_speech	0.085184498	0.04619861	-0.330623200	-0.278608081
raise_min_wage	0.101928809	0.25895530	-0.487814500	0.519465060
govt_reg_social_media	-0.681233929	0.01893208	-0.085293073	0.110874543
limit_immigration	-0.043706839	-0.56351946	-0.093304242	0.028332979
border_security_funding	-0.044875812	0.57779142	0.002353579	-0.039266546

	PC9	PC10
same_sex_marriage	-0.650913728	0.16084674
progressive_taxation	0.162671438	-0.35532427
increase_natl_def	-0.057447062	0.10156644
tuition_free_edu	0.103530913	0.42360062
access_abortion	0.323171313	0.44947285
restrict_hate_speech	0.347570746	-0.58502662
raise_min_wage	-0.269166976	-0.09673095
govt_reg_social_media	-0.003036614	-0.04172217
limit_immigration	-0.345178837	-0.23011263
border_security_funding	0.344930245	0.22313300

Looking at the $\widehat{\text{PCs}}$

For the first 10 observations:

	PC1	PC2	PC3	PC4	PC5	PC6
1	-0.7113033	2.4408545	-1.2077156	-1.88518673	0.83255370	1.55504613
2	0.2035785	2.8977192	-0.1402032	-1.26239661	-0.06643766	-1.34499893
3	-2.1718272	-2.7964371	1.7331104	2.13638812	-0.58238668	-0.75171374
4	-1.1829479	2.1153775	-0.9585381	1.11144253	1.25639455	-0.95760962
5	1.8367283	-0.1114780	-0.1437611	1.56243729	0.68321580	-0.01765701
6	1.1554626	-0.4004424	1.1040280	1.11212103	0.02769744	-0.01345777
7	-2.2289246	-0.9158334	1.5646457	0.78506734	0.48905941	-0.88603099
8	-1.2871974	0.3965876	0.1096556	0.01164672	-1.11441939	0.77705967
9	-2.5948455	-1.2903976	-1.5126712	0.17952102	-1.03362097	0.04259595
10	0.1770839	-0.9178399	-0.8072962	1.03317014	0.04626742	-0.81063825

	PC7	PC8	PC9	PC10
1	-1.76679474	-0.5301101	-0.54444130	0.01709152
2	0.06808724	-0.5416930	-1.90867206	-0.51570280
3	0.23961941	-0.2501476	0.14243160	-1.70692631
4	-0.14123007	-0.1842250	-0.58239819	-0.16742082
5	-1.66042718	-0.5525442	0.13990851	-0.29673893
6	-0.07647689	0.1642122	-1.20810523	-1.64437951
7	-0.20825566	-0.1959137	-0.57875981	-0.27502801
8	-0.54508906	0.3014855	-1.32784851	-0.07235607
9	-0.05549644	-0.1450357	0.06500219	-0.09506057
10	-1.18094875	-0.1533043	-0.77096637	0.19688378

Choosing PCs: looking at variance captured

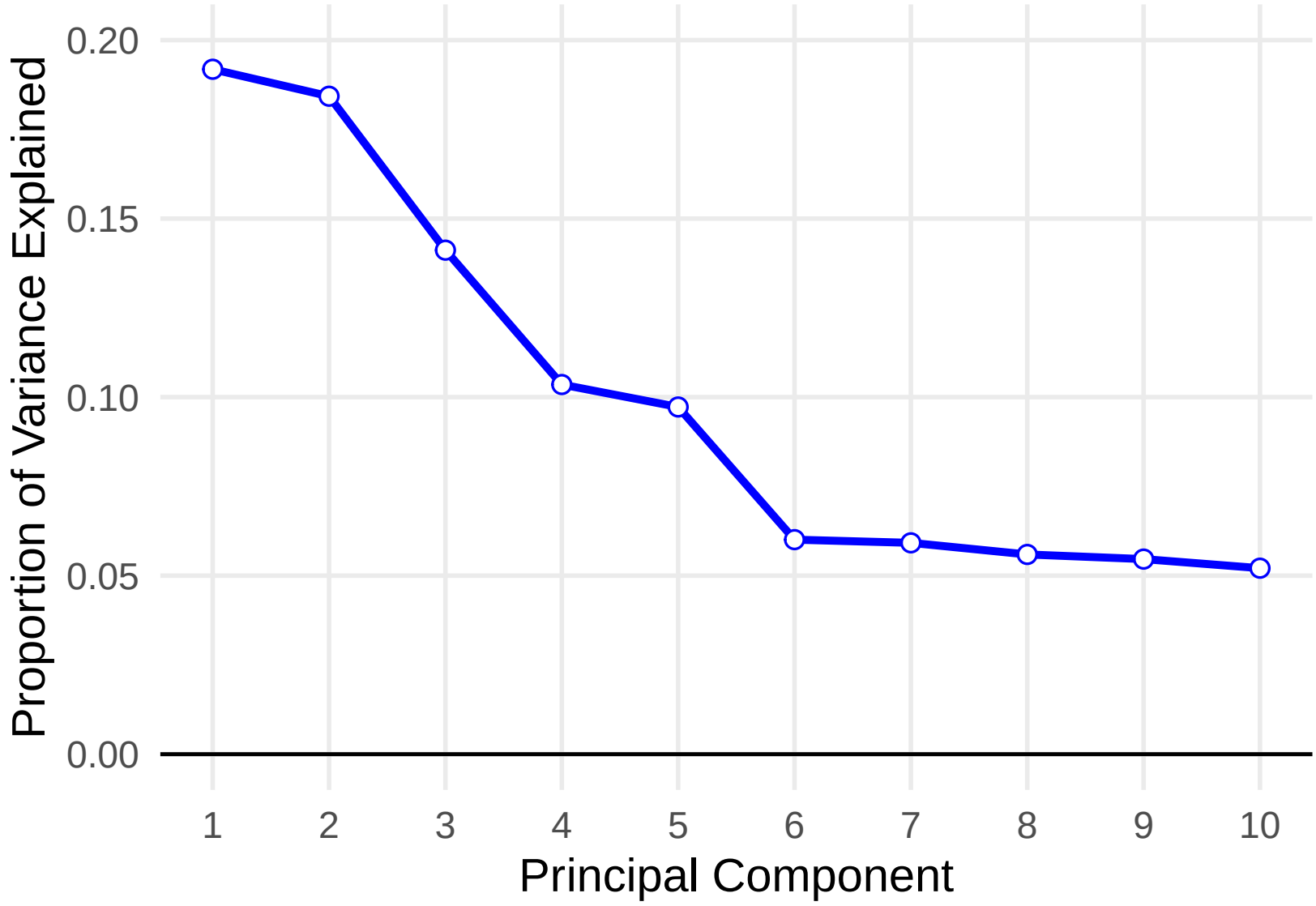
How much of the variance does each PC capture?

```
summary(pca_result)$importance[2:3, ]
```

```
                            PC1     PC2     PC3     PC4     PC5     PC6     PC7
Proportion of Variance 0.19183 0.18428 0.14117 0.10353 0.09724 0.06009 0.05919
Cumulative Proportion  0.19183 0.37611 0.51727 0.62080 0.71805 0.77813 0.83732
                            PC8     PC9    PC10
Proportion of Variance 0.05594 0.05464 0.05209
Cumulative Proportion  0.89327 0.94791 1.00000
```

Choosing PCs: elbow plots

We can plot this in an **elbow plot** (aka **scree plot**).



Clustering methods

The idea of "clusters"

A **cluster** is a group of observations that are very similar to each other, but very different from observations outside the cluster.

Clustering is "unsupervised classification", so:

- ➜ Clusters do not relate features to pre-existing classes.
- ➜ You estimate membership in *pre-determined number of groups* (yikes!).

Groups formed by clusters are given labels through *post-estimation interpretation of their elements* (yikes!).

These methods are typically used when we do not and never will know the "true" class labels.

K-means clustering

Very common method for clustering: **K-means clustering**

The basic idea:

- ➜ Each observation is assigned to one of K clusters
- ➜ Assignment of observations to clusters occurs in a way that minimises within-cluster difference and maximises between-cluster differences
- ➜ Uses random starting positions and iterates until stable
- ➜ Treats feature values as coordinates in a multidimensional space

K-means clustering

Advantages

- ➜ simple
- ➜ highly flexible
- ➜ efficient

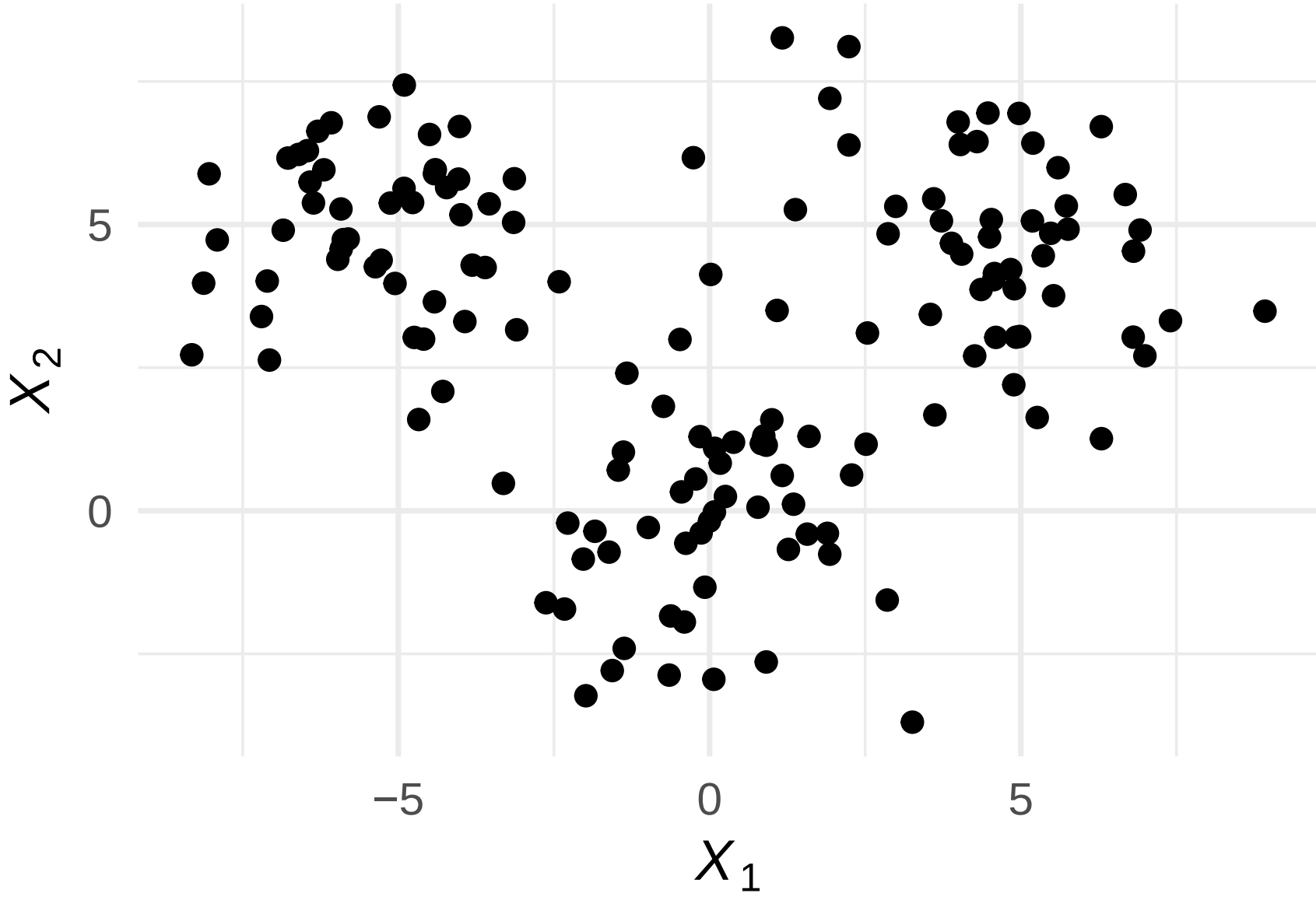
Disadvantages

- ➜ no fixed rules for determining K
- ➜ uses an element of randomness for starting values, so can get different clusters each time you run the algorithm!

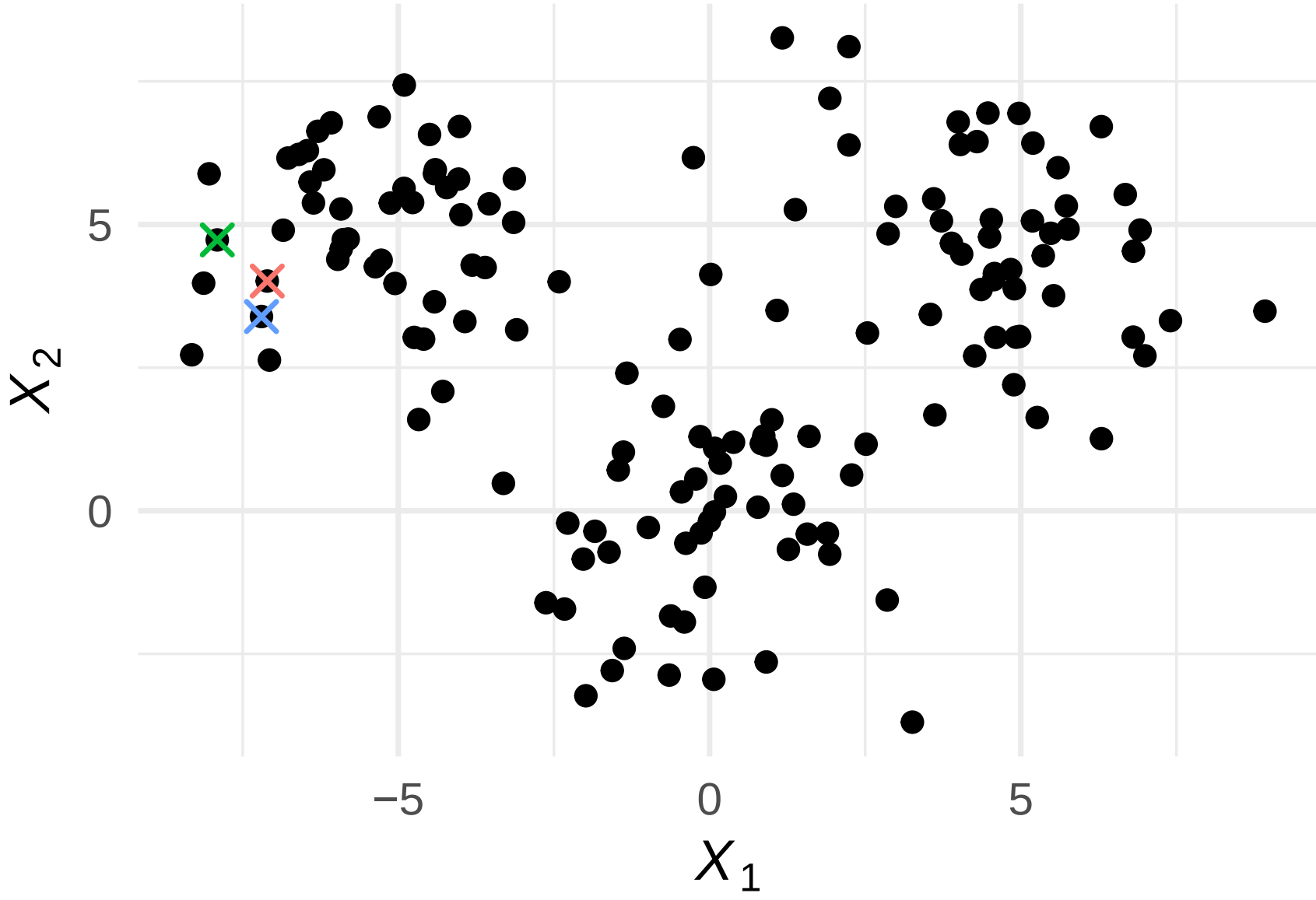
Algorithm details

1. Choose starting values
   - ➜ Assign random positions to K starting values that will serve as the "cluster centres", known as "centroids"; or,
   - ➜ Assign each observation randomly to one of K classes
2. Assign each item to the class of the centroid that is "closest"
   - ➜ Euclidean distance is most common, but others can be used
3. Update: recompute the cluster centroids as the mean value of the points assigned to that cluster
4. Repeat reassignment of points and updating centroids
5. Repeat 2–4 until some stopping condition is satisfied
   - ➜ e.g. when no items are reclassified following update of centroids

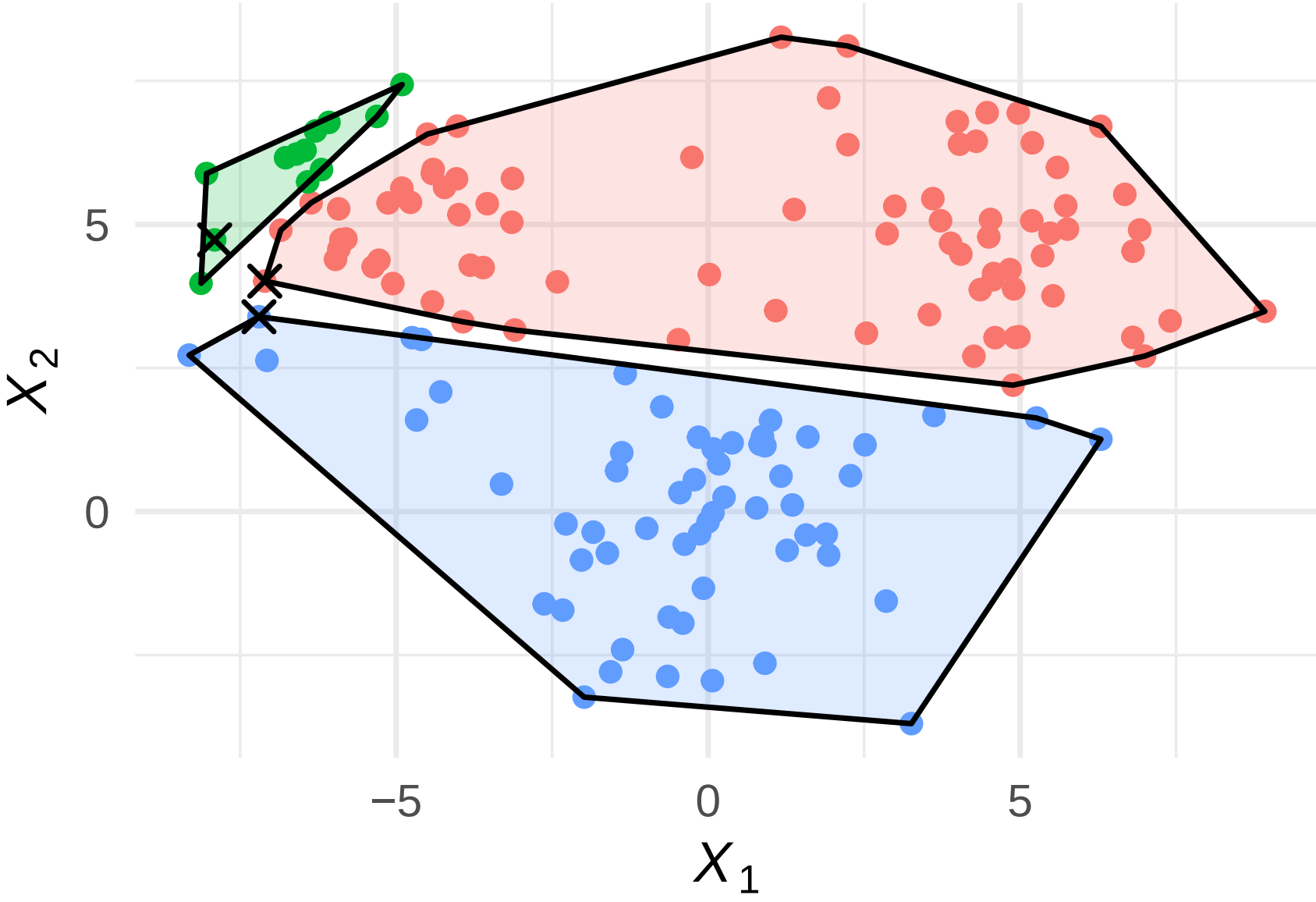
K-means clustering illustrated



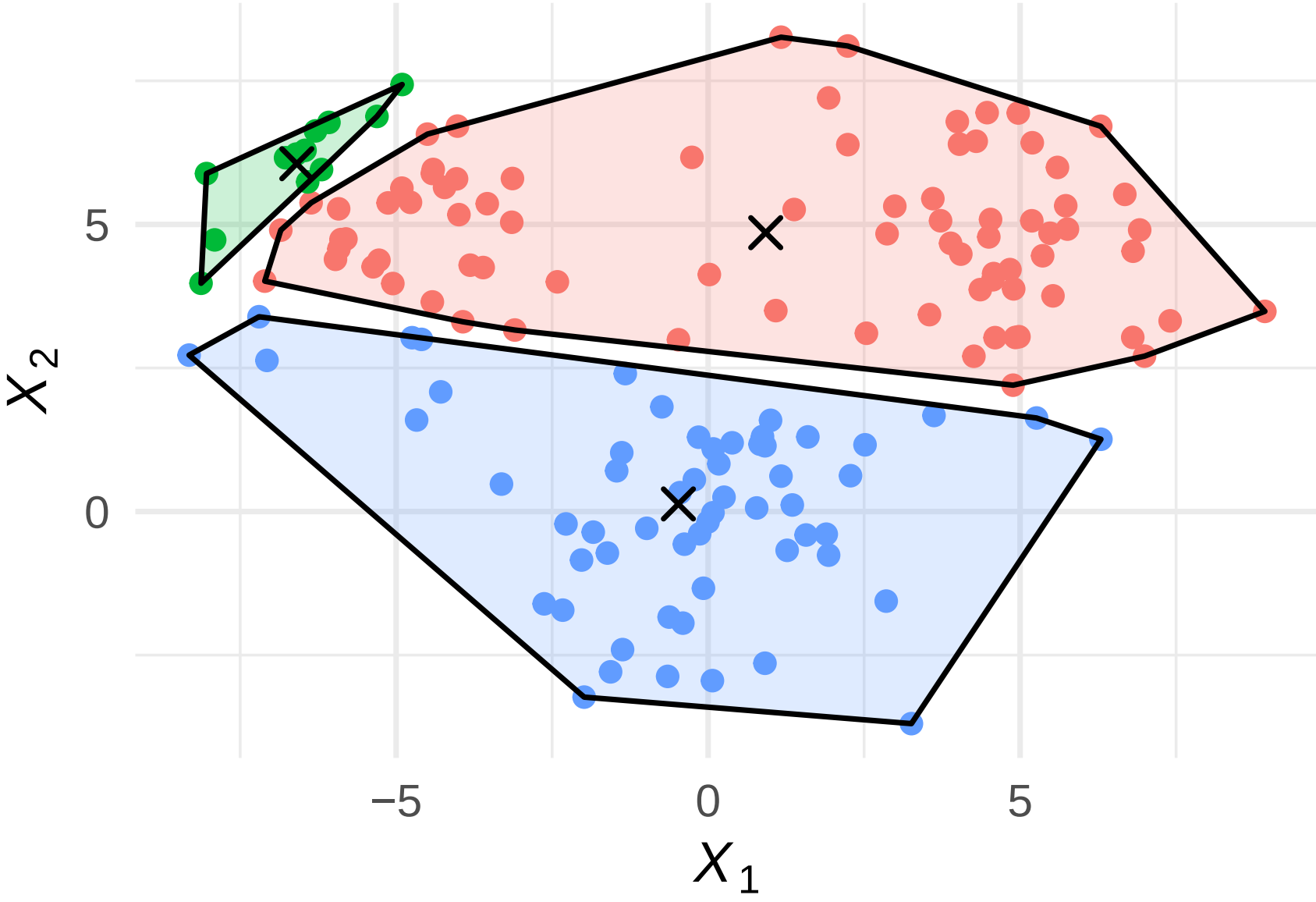
K-means clustering illustrated



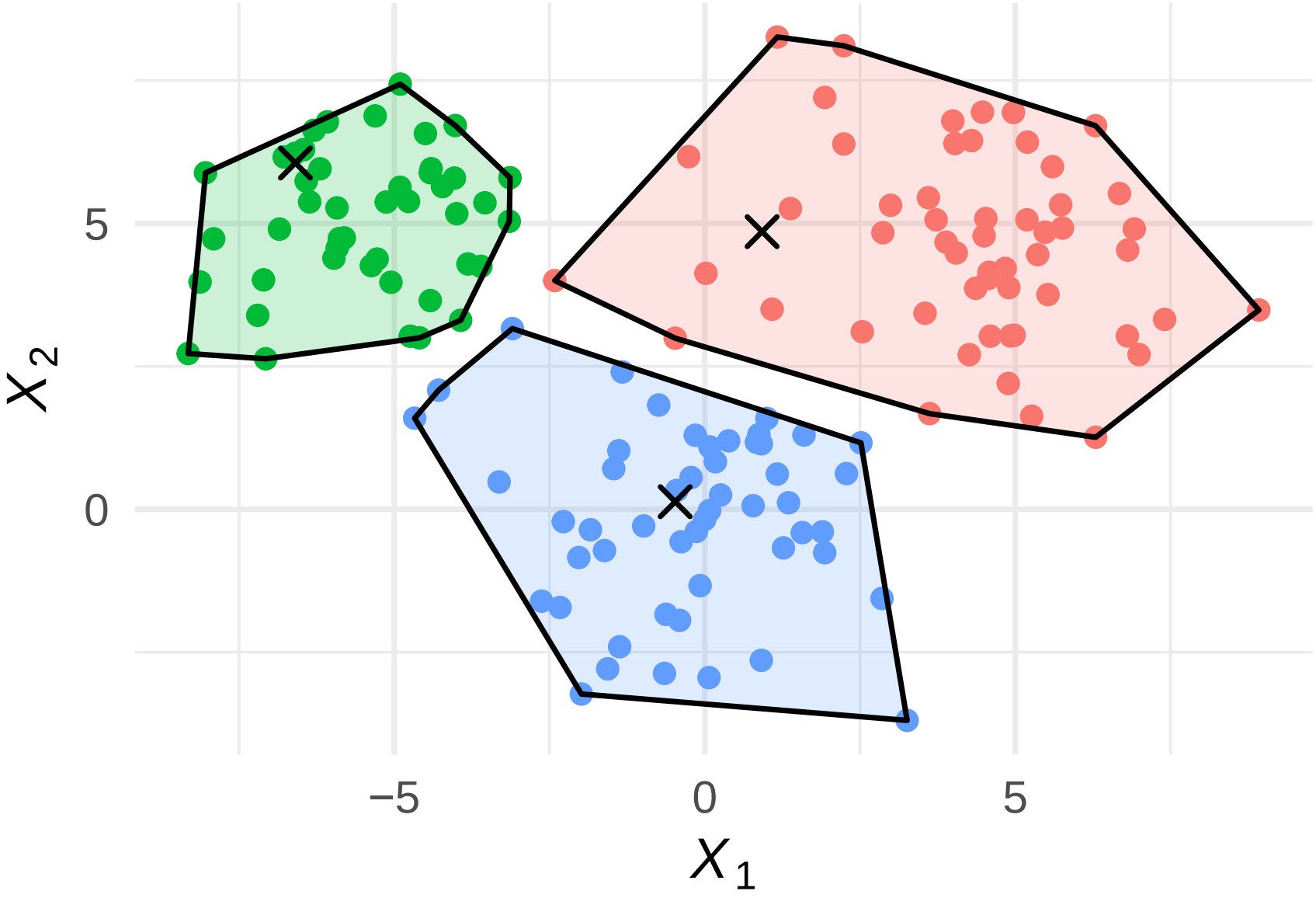
K-means clustering illustrated



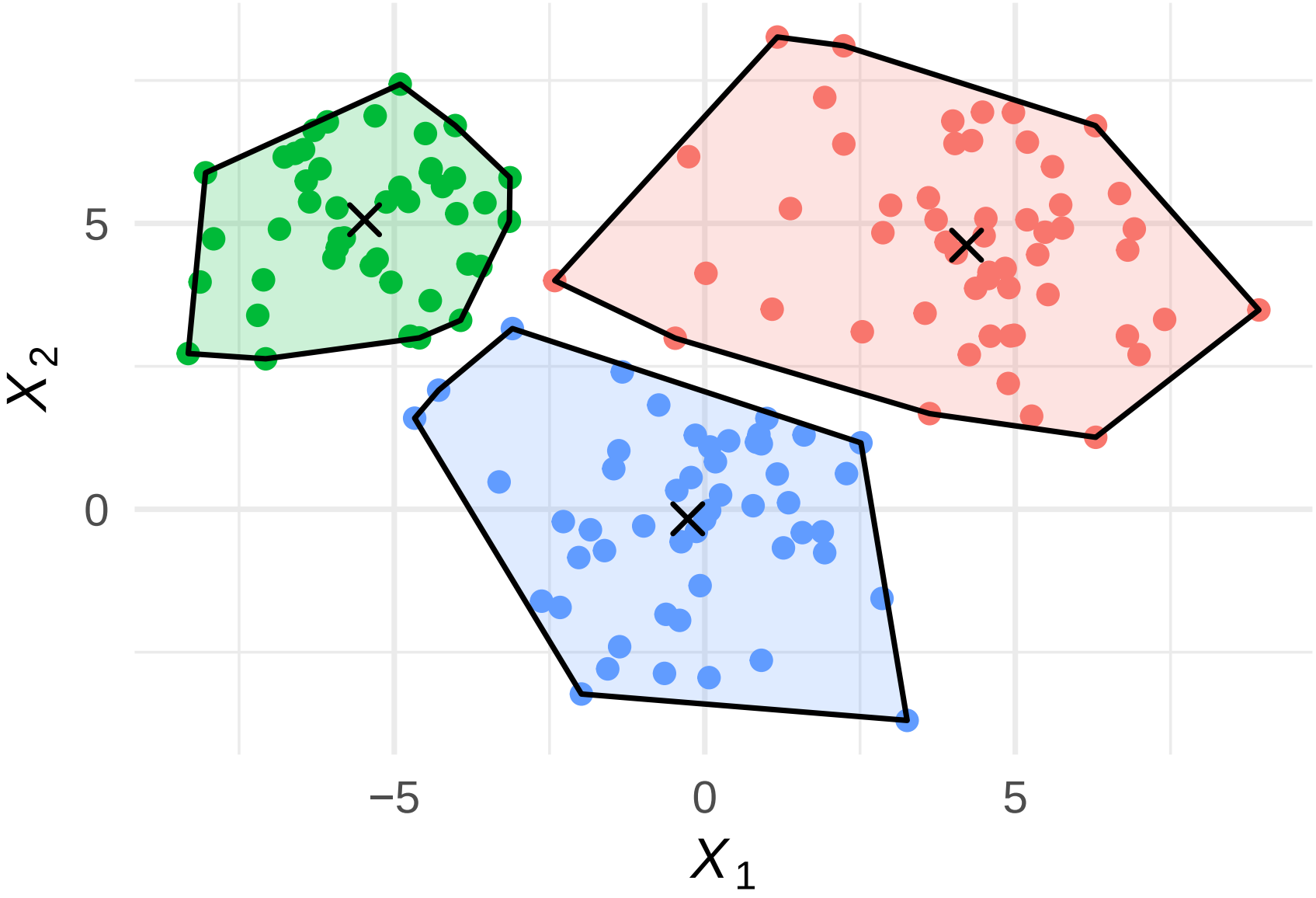
K-means clustering illustrated



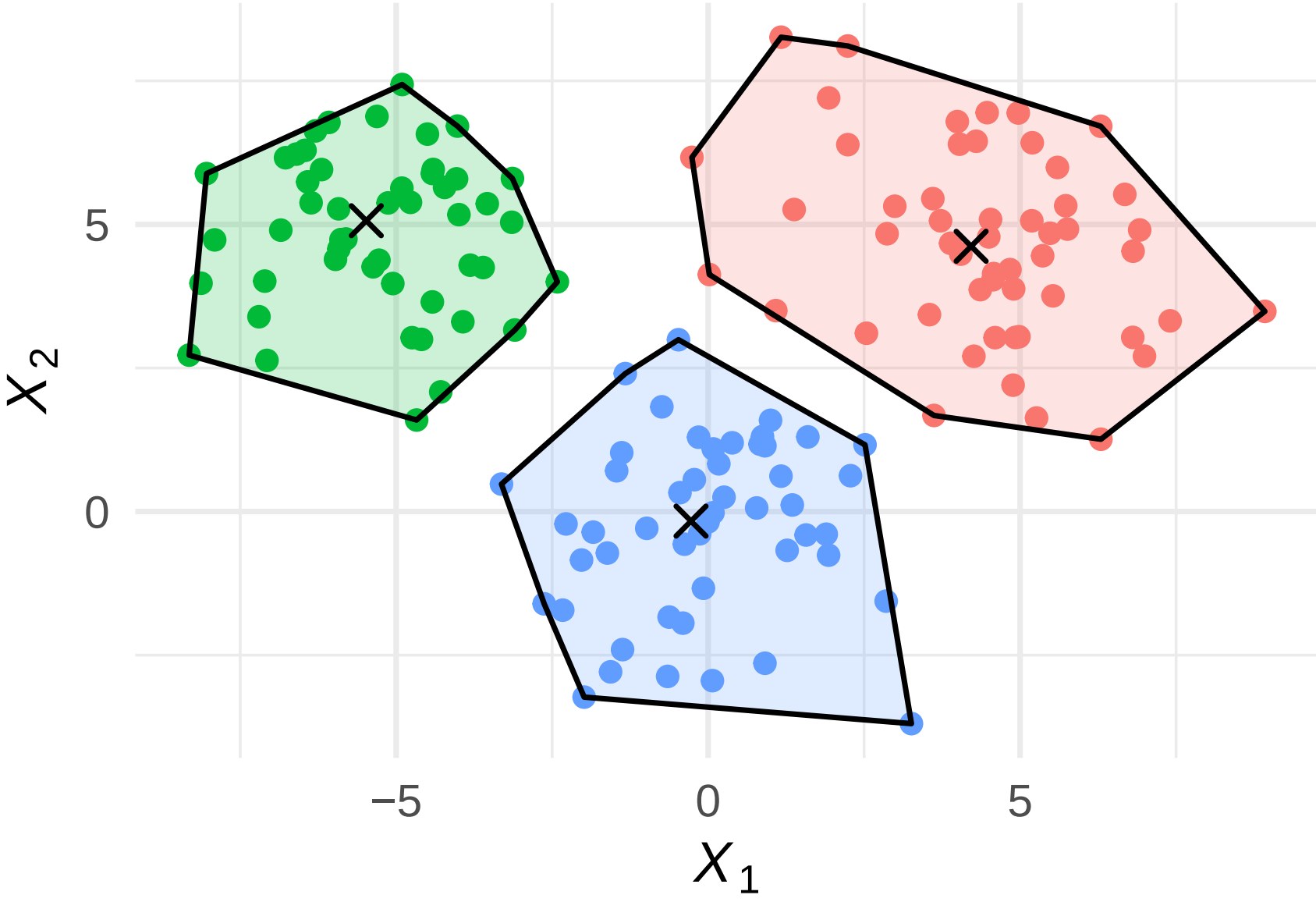
K-means clustering illustrated



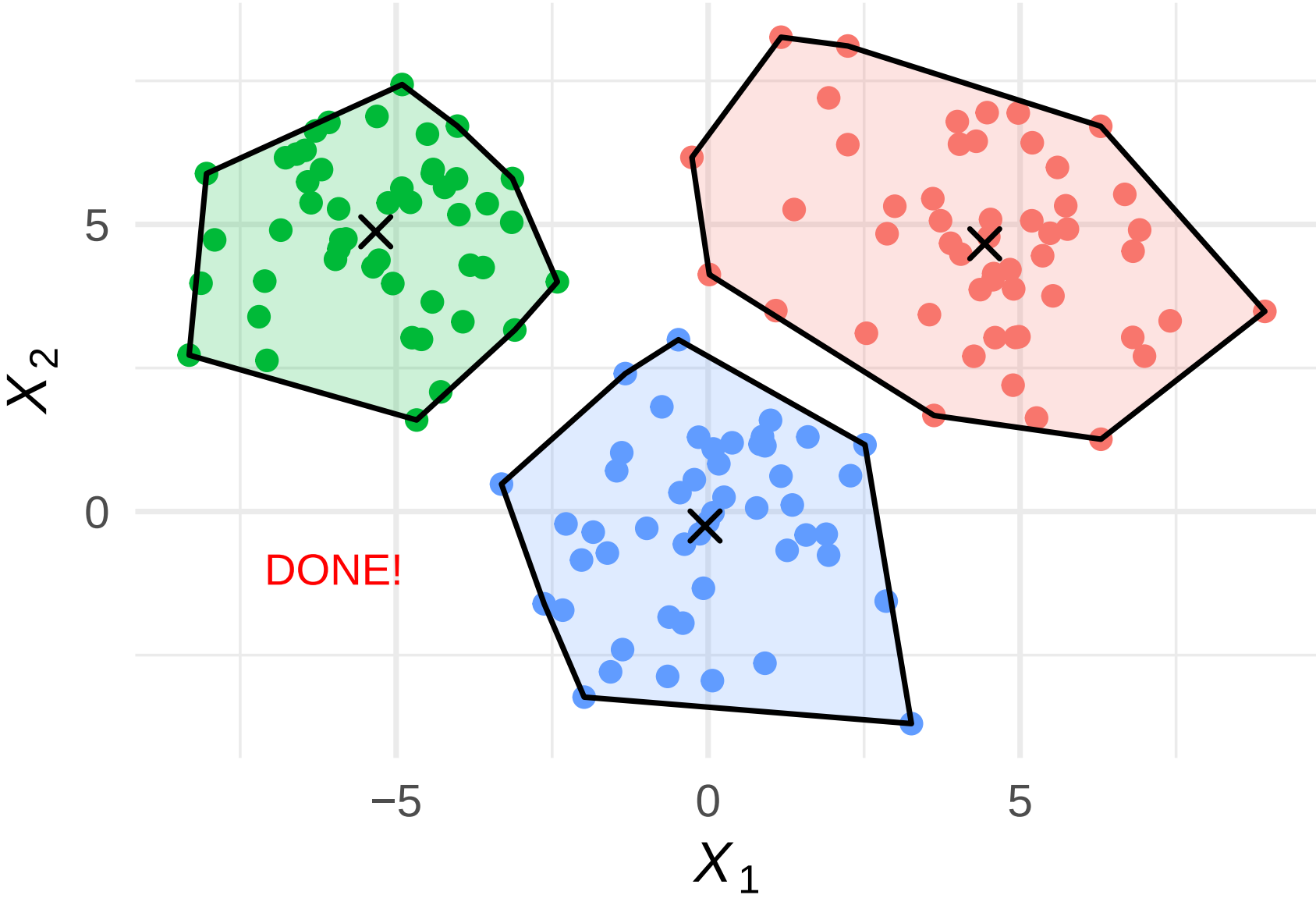
K-means clustering illustrated



K-means clustering illustrated



K-means clustering illustrated



K-means clustering illustrated

In R, this all happens under the hood.

```
km <- kmeans(df, centers = 3)
print(km)
```

```
K-means clustering with 3 clusters of sizes 50, 51, 49

Cluster means:
             x          y
1 -0.04756148 -0.1642821
2  4.51983757  4.6730255
3 -5.32672081  4.8720163

Clustering vector:
  [1] 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [38] 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2
 [75] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3
[112] 3 3 3 3 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
[149] 3 3

Within cluster sum of squares by cluster:
[1] 241.6745 292.1295 187.5289
 (between_SS / total_SS =  81.8 %)

Available components:
```

K-means clustering illustrated

Remember: K-means usually starts by randomly selecting starting clusters.

- ➜ Clusters could be different based on starting values.
- ➜ `R` allows you to specify `nstart`: how many times do you want to run the algorithm with different start values? (Picking best one.)

```
km <- kmeans(df, centers = 3, nstart = 1)
print(km$tot.withinss)
```

```
[1] 721.3328
```

```
km <- kmeans(df, centers = 3, nstart = 25)
print(km$tot.withinss)
```

```
[1] 721.3328
```

Choosing the appropriate number of clusters

Major point of analyst discretion: how many clusters.

- Very often based on prior information about the number of categories sought.
  - for example, you need to cluster people in a class into a fixed number of (like-minded) tutorial groups.
- A (rough!) guideline: set $K = \sqrt{N/2}$ where N is the number of observations to be clustered.
  - usually too big: setting K too large will improve within-cluster similarity, but risks *overfitting*.
- Again, use elbow plots: fit multiple clusters with different K values, and choose the K beyond which there are diminishing gains.

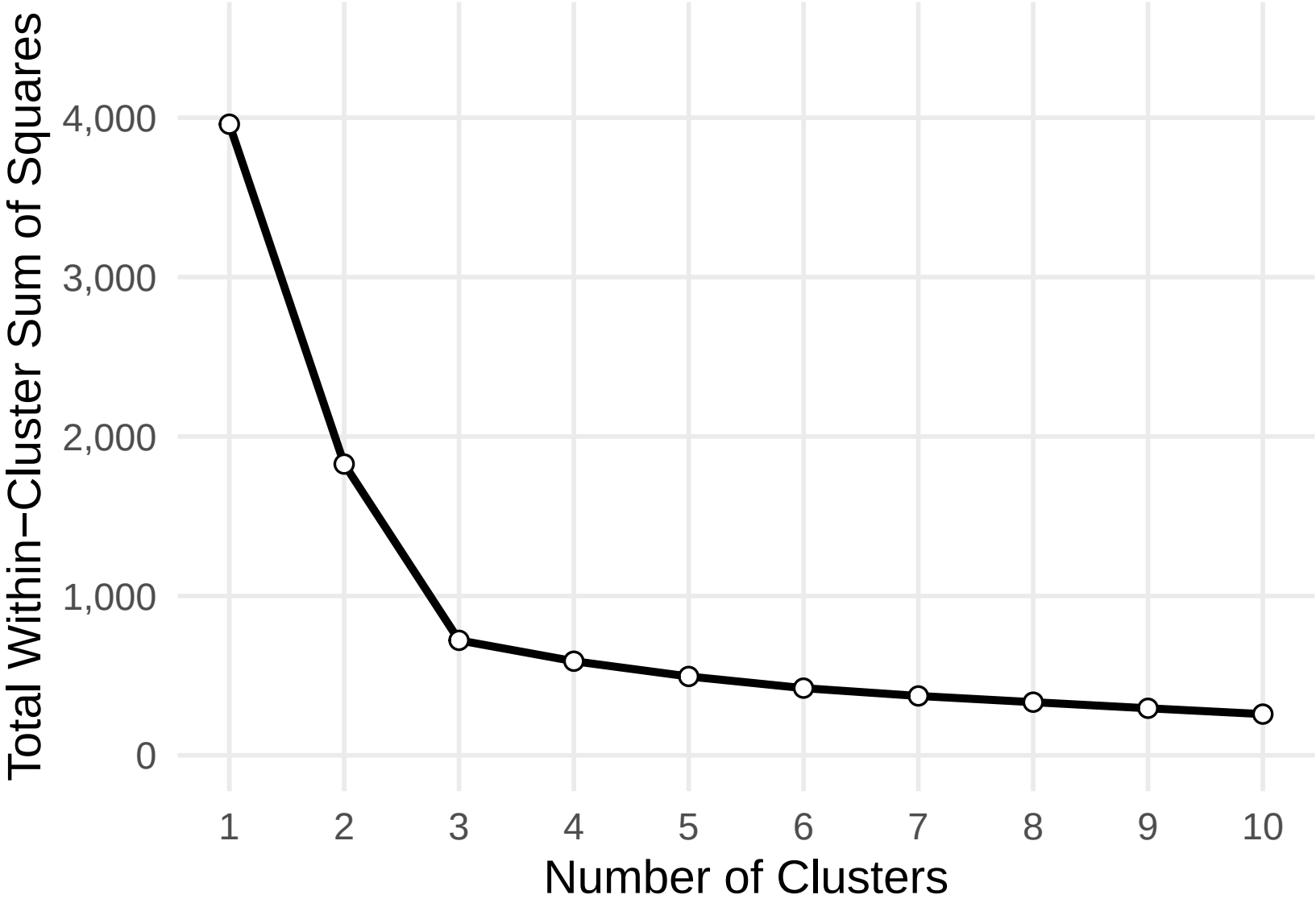
Choosing the appropriate number of clusters

```r
# Range of k values to try
k_values <- 1:10

elbow.plot <- unlist(lapply(k_values, function(x) kmeans(df, centers = x)$tot.withinss))
elbow.plot <- bind_cols(k = k_values, z = elbow.plot)

ggplot(elbow.plot, aes(x = k, y = z)) +
  geom_line(aes(group = 1), color = "black", linewidth = 2) +
  geom_point(size = 4, shape = 21, fill = "white", stroke = 1) +
  scale_x_continuous(limits = c(min(k_values), max(k_values)),
                     breaks = seq(min(k_values), max(k_values),1)) +
  scale_y_continuous(limits = c(0, 4500),
                     breaks = seq(0,4500,1000),
                     labels = scales::comma(seq(0,4500,1000))) +
  labs(x = "Number of Clusters", y = "Total Within-Cluster Sum of Squares") +
  theme_minimal(base_size = 25) +
  theme(panel.grid.minor = element_blank())
```

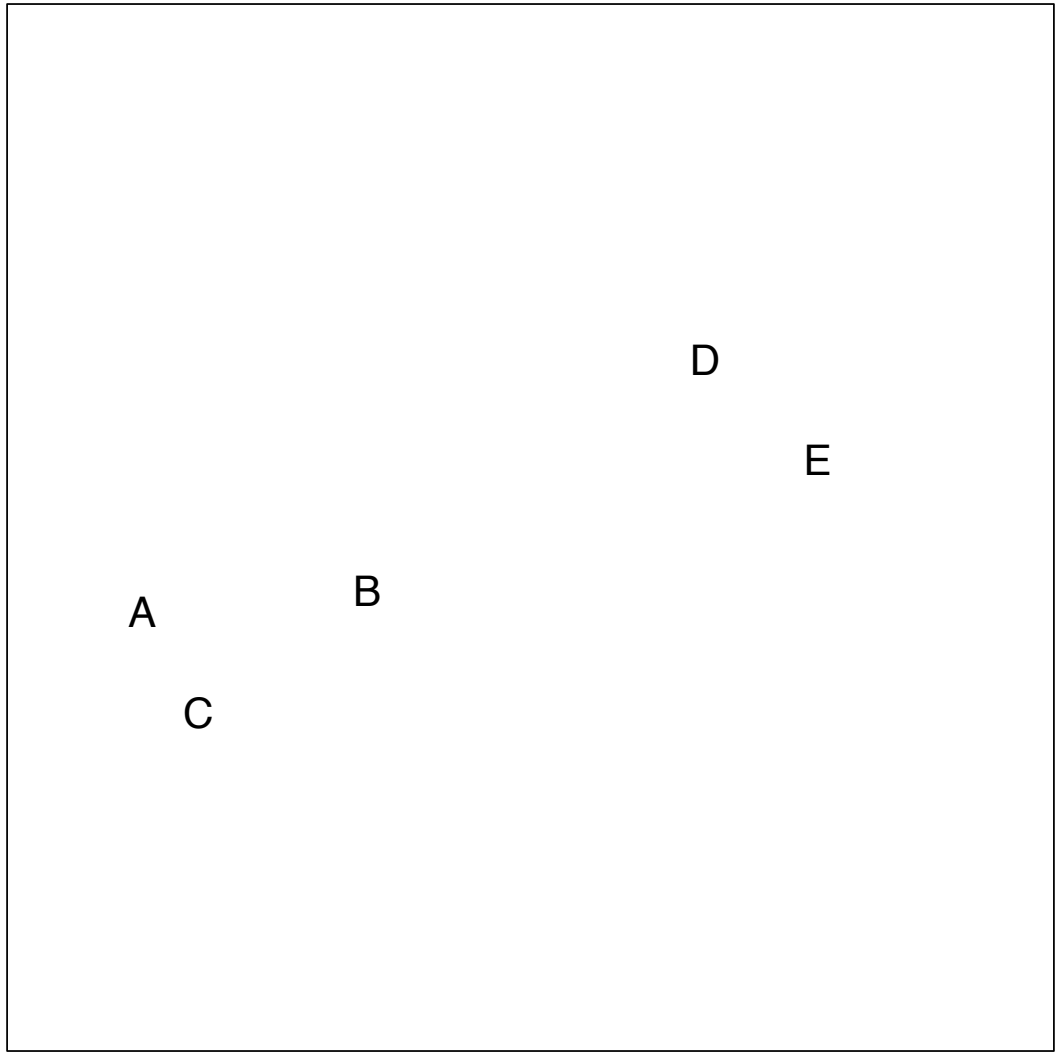
Choosing the appropriate number of clusters



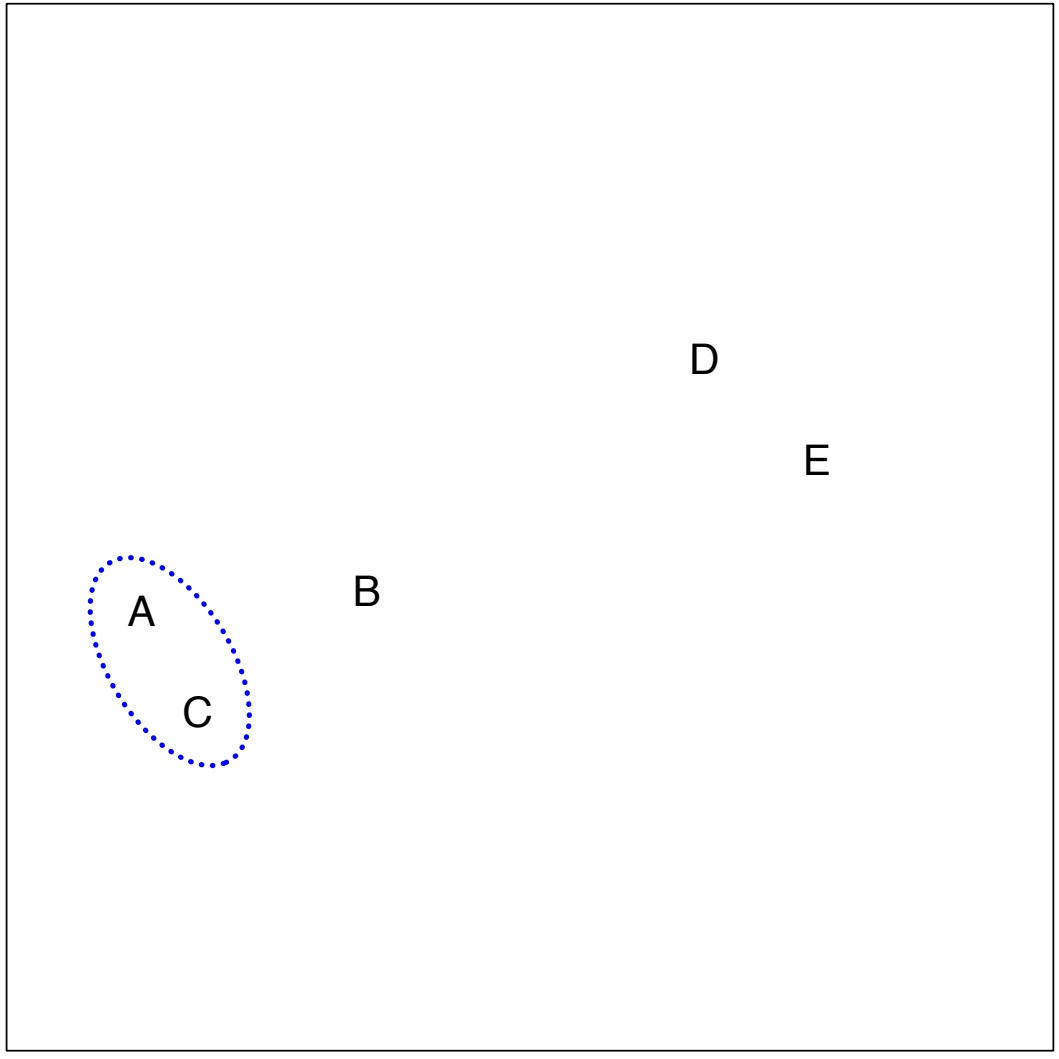
Hierarchical clustering

- For K-means clustering, have to pre-specify $\#$ of clusters K.
- Hierarchical clustering is an alternative approach which does not require that we commit to a particular choice of K.
- There are a lot of methods for hierarchical clustering; we'll focus on **bottom-up** or **agglomerative** clustering.
  - This is the most common type of hierarchical clustering.
  - A **dendrogram** is built starting from the leaves and combining clusters up to the trunk.
- Still need to choose a number of clusters at some point (eek), but here, it's post estimation.
- (This is a tree method for clustering!)

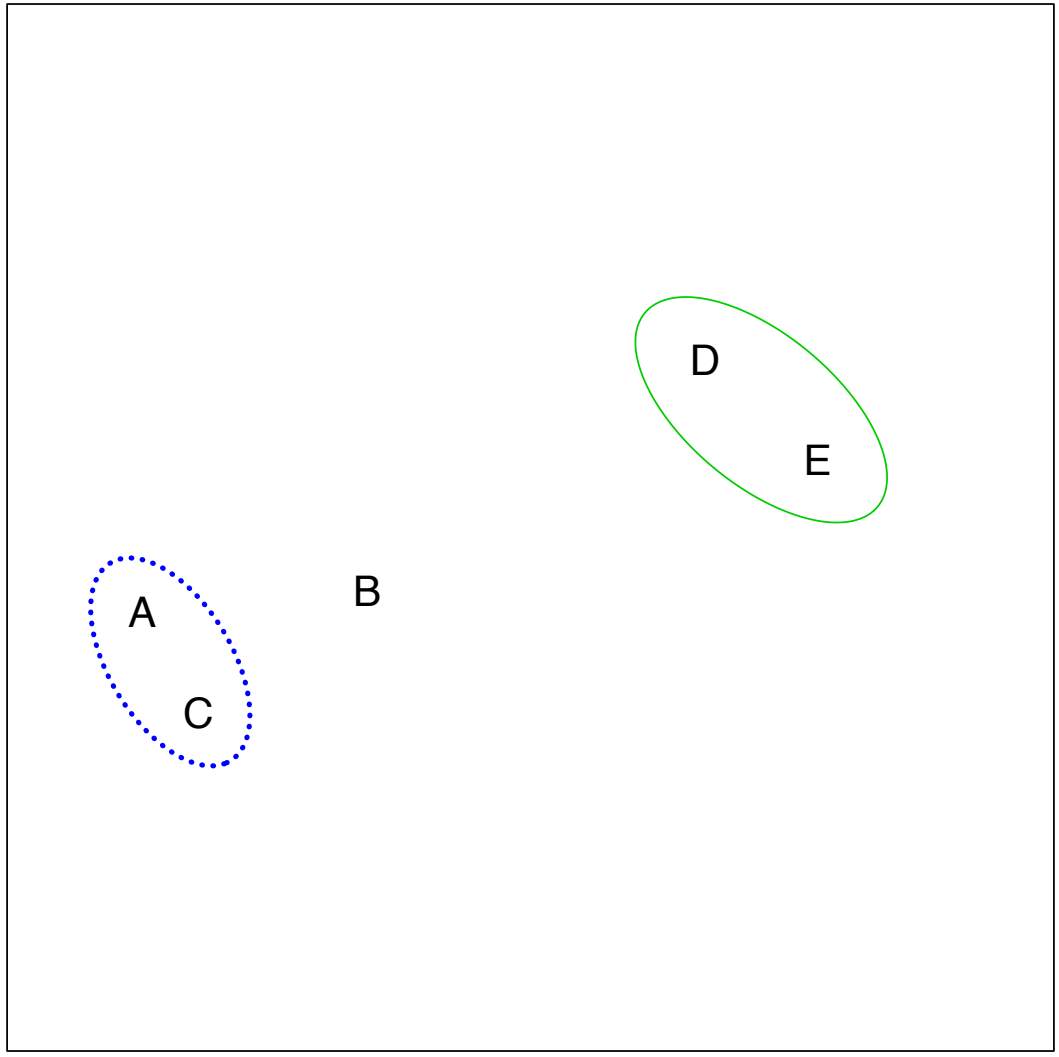
Hierarchical clustering: the idea



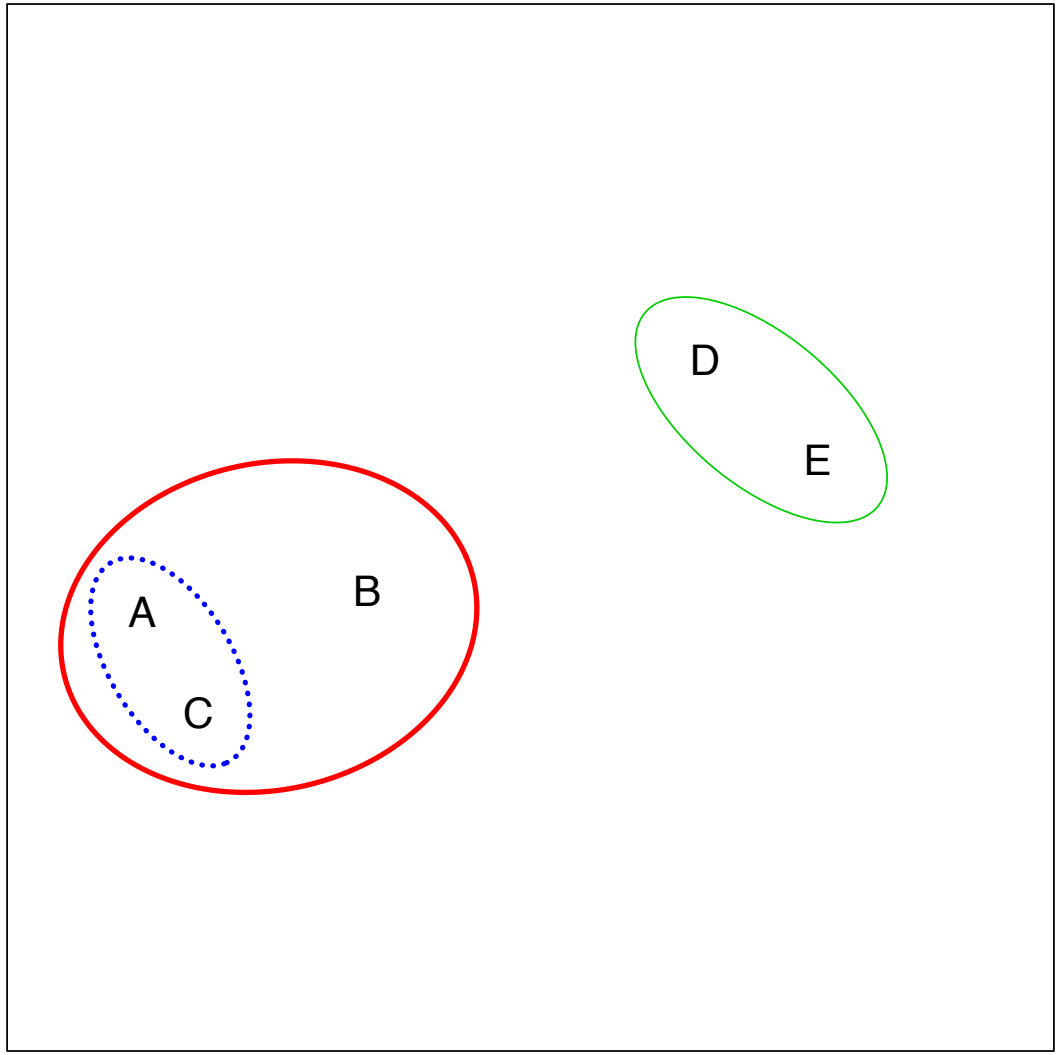
Hierarchical clustering: the idea



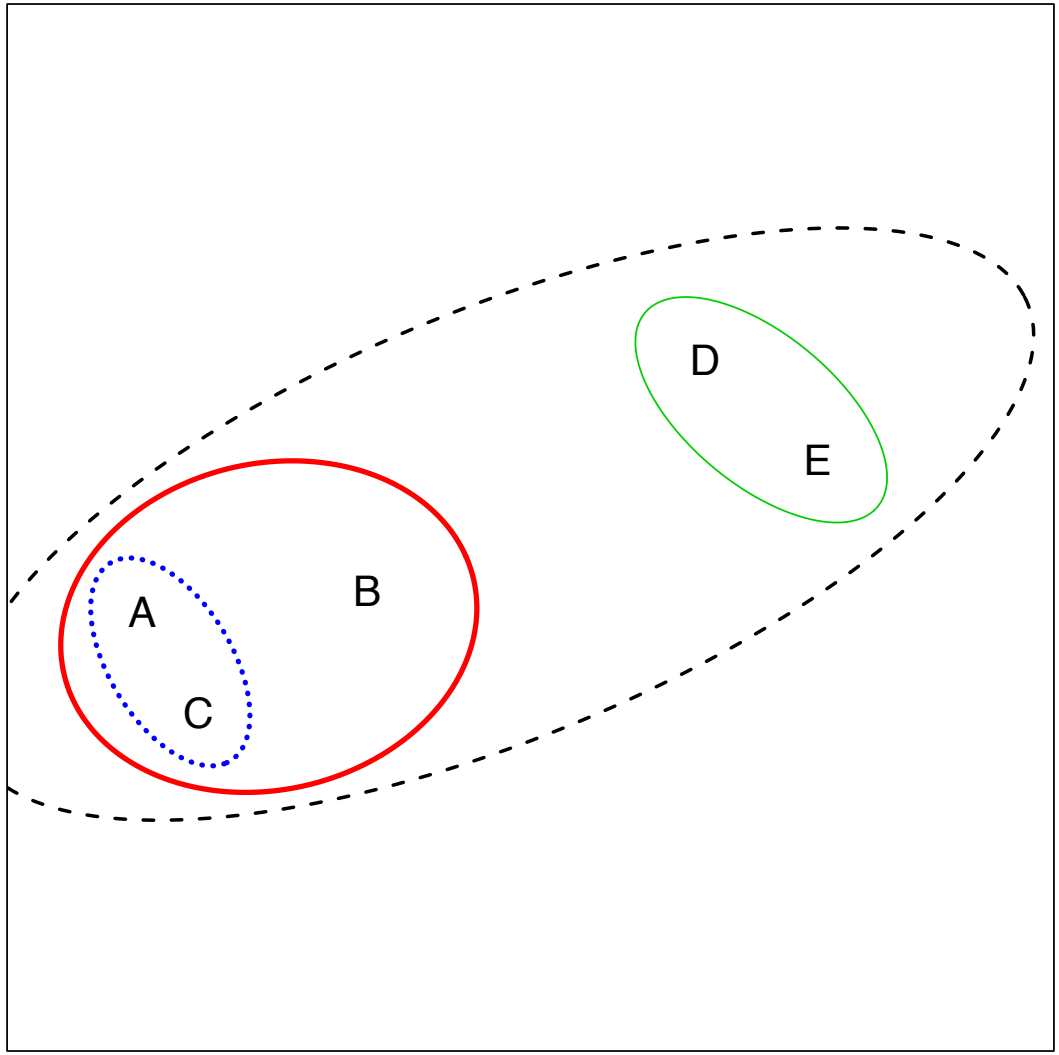
Hierarchical clustering: the idea



Hierarchical clustering: the idea



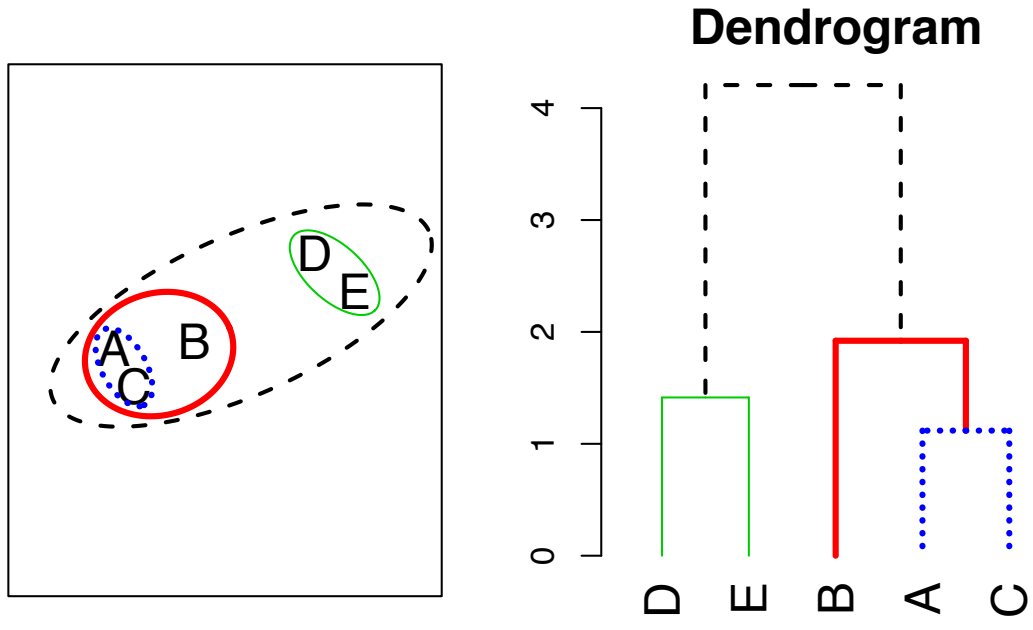
Hierarchical clustering: the idea



Hierarchical clustering algorithm

The approach in words:

1. Start with each point in its own cluster.
2. Identify the closest two clusters and merge them.
3. Repeat.
4. Ends when all points are in a single cluster.



Hierarchical clustering algorithm

After you have finished this, you can create a **dendrogram**

This has a tree-like structure

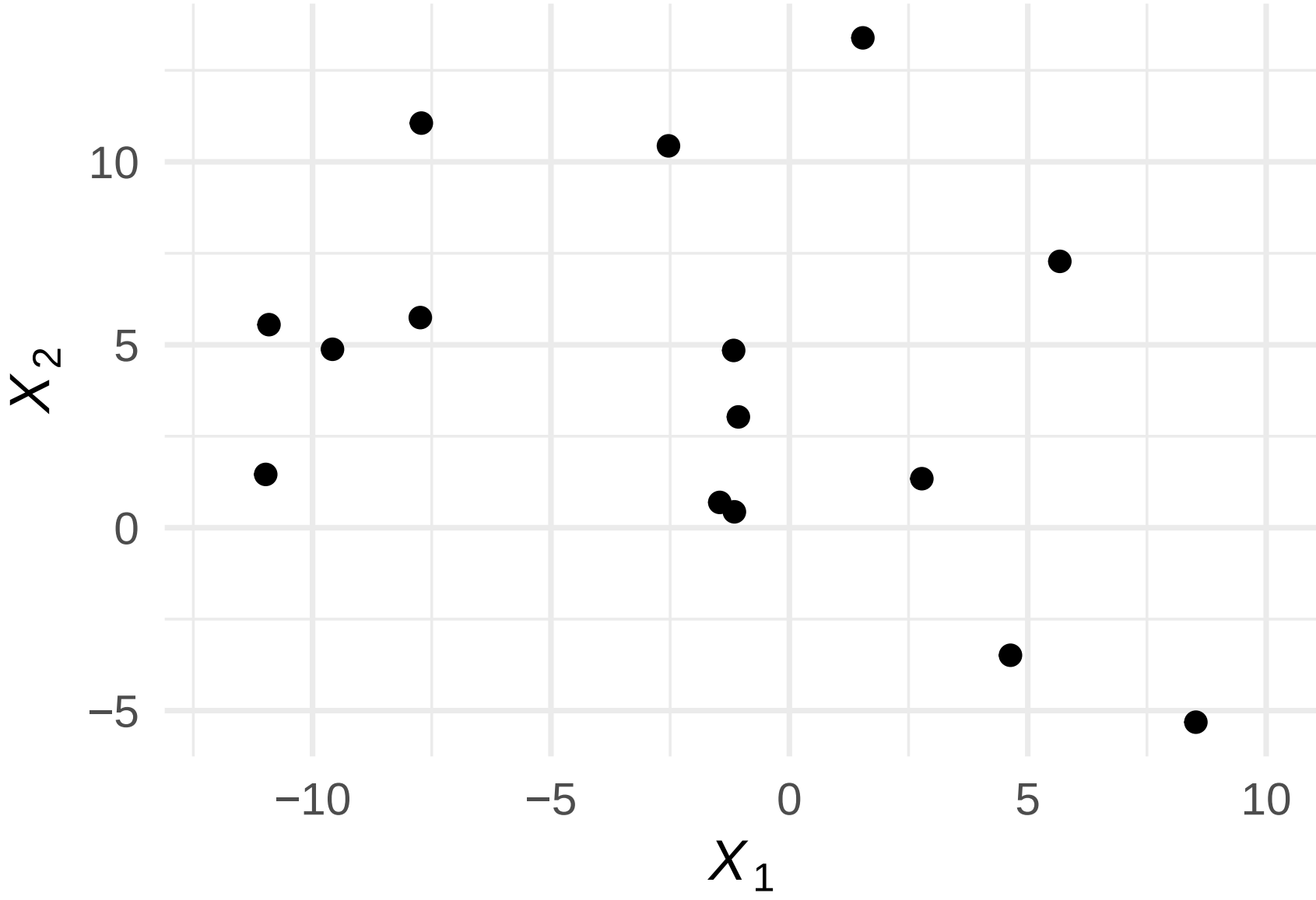
You can "cut" it where you want—i.e., how many clusters do you want to consider

There isn't a unique ordering of the leaves of a dendrogram, so making it look nice and interpretable can be tricky

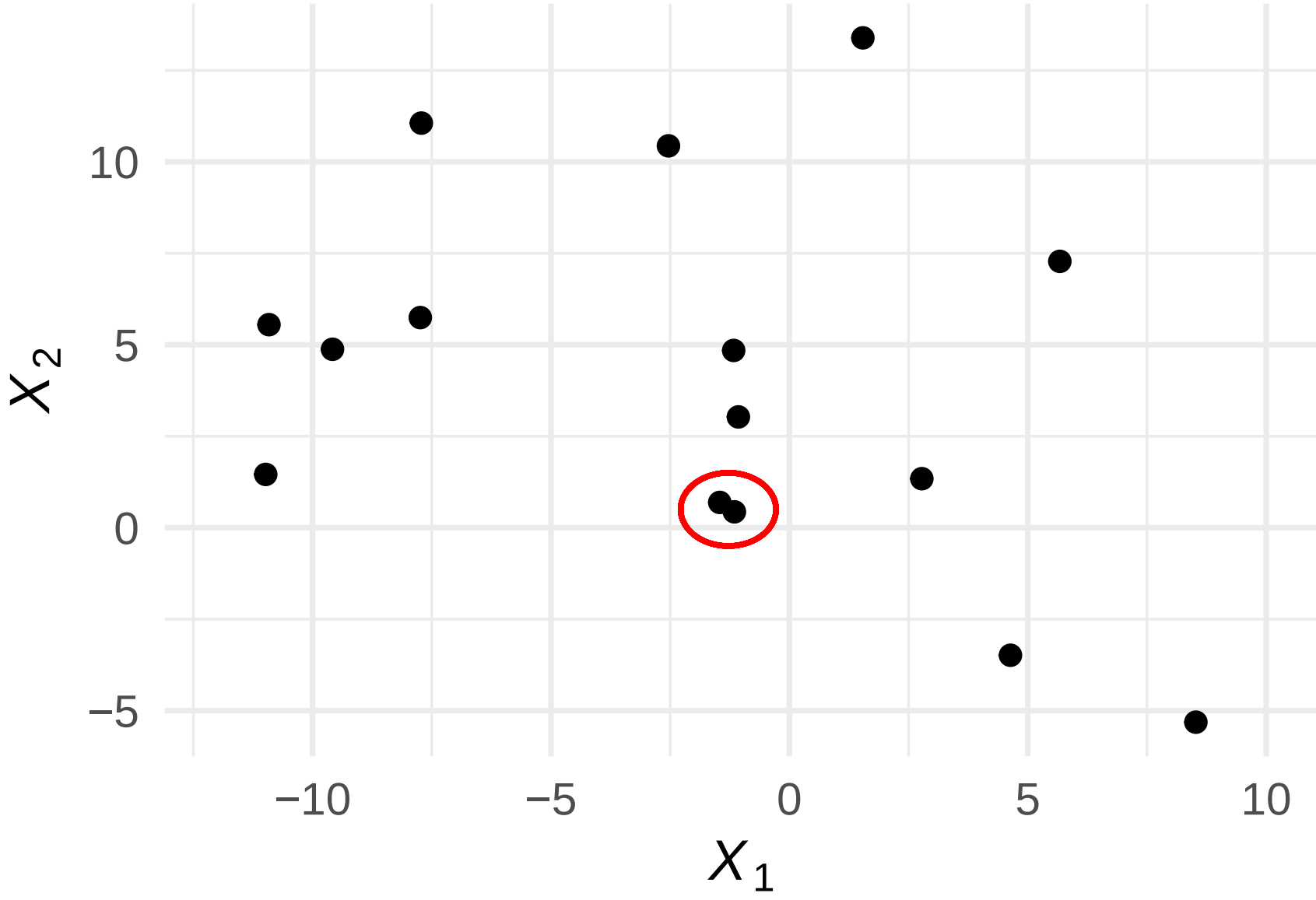
Example dendrogram

	X1	X2
1	-1.069615	3.0273480
2	-10.913514	5.5494041
3	-1.460217	0.6909044
4	4.637029	-3.4865115
5	2.778377	1.3390973
6	5.672380	7.2793425
7	1.541631	13.3882332
8	8.525410	-5.3159345
9	-1.152950	0.4334490
10	-1.168036	4.8445117
11	-2.535241	10.4362248
12	-7.718248	11.0586423
13	-10.981857	1.4560619
14	-9.581384	4.8761669
15	-7.739992	5.7426296

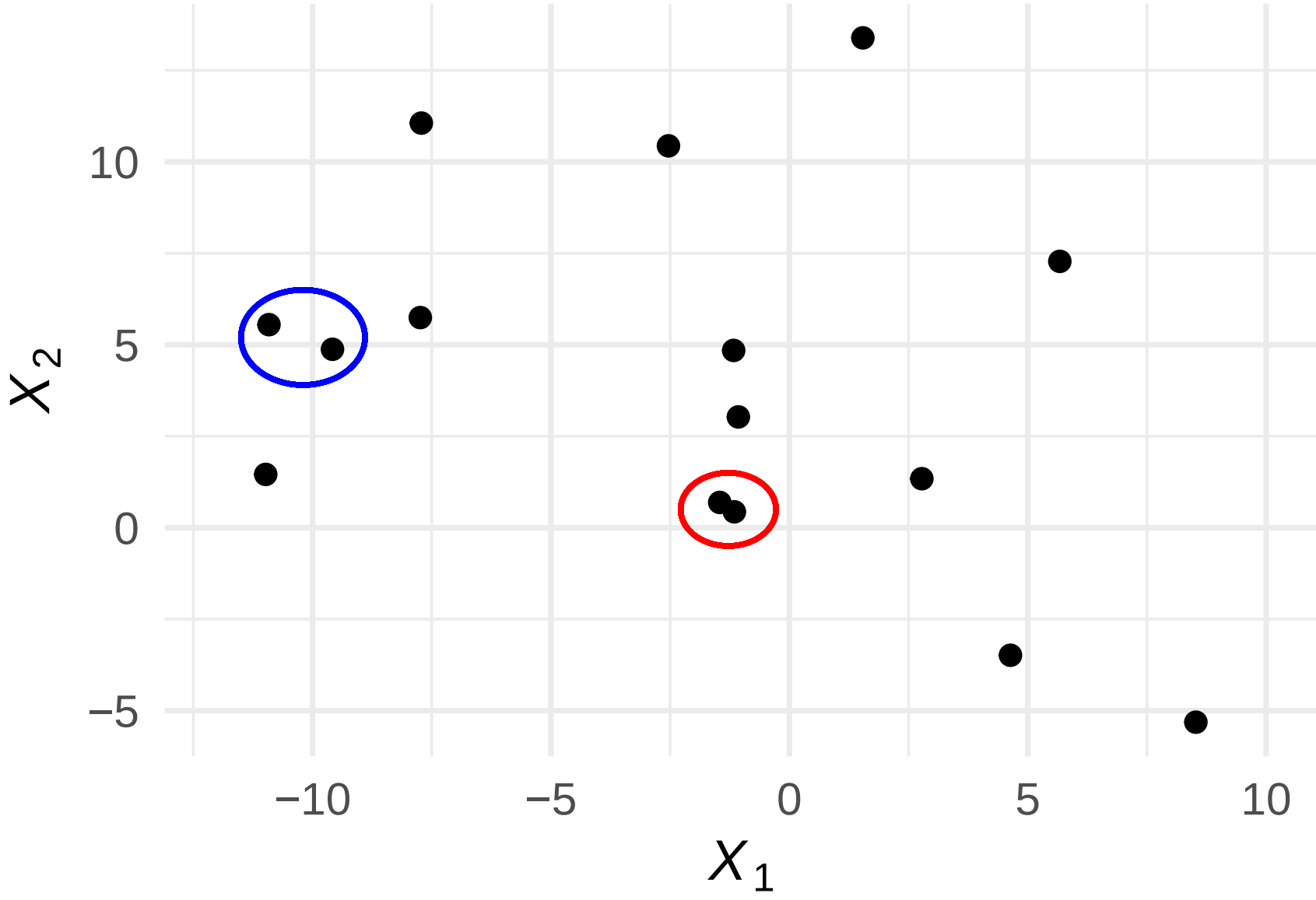
Example dendrogram



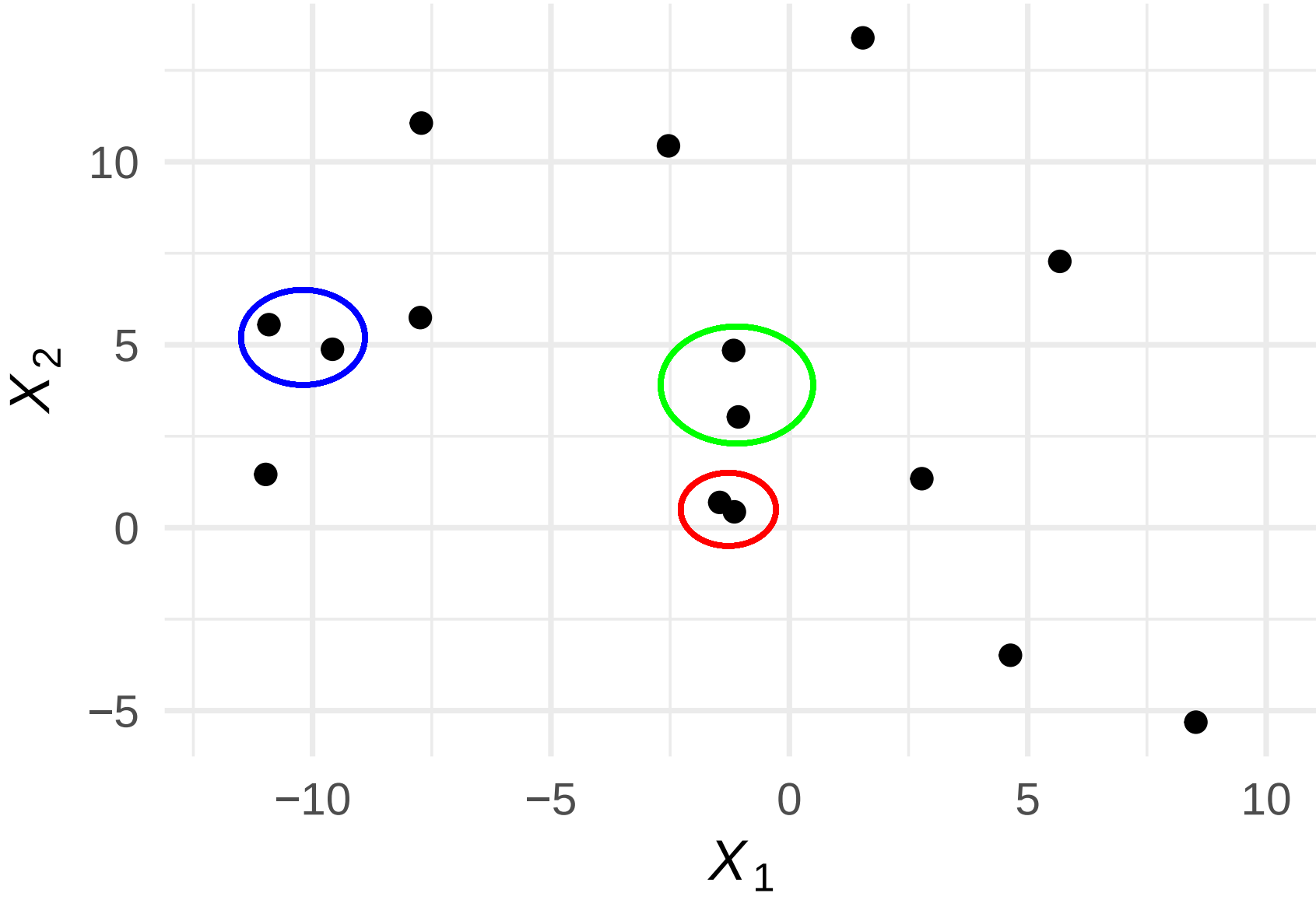
Example dendrogram



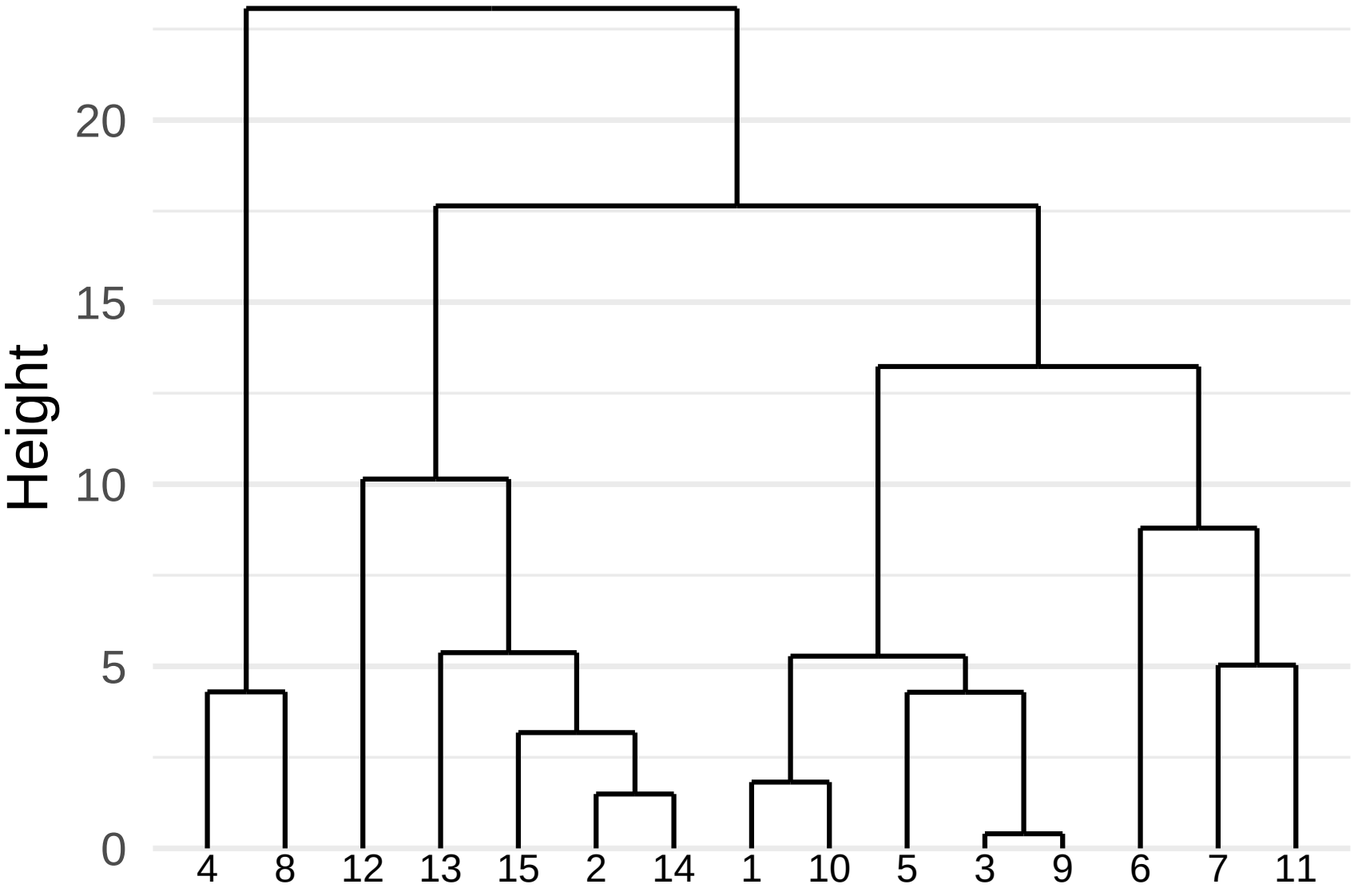
Example dendrogram



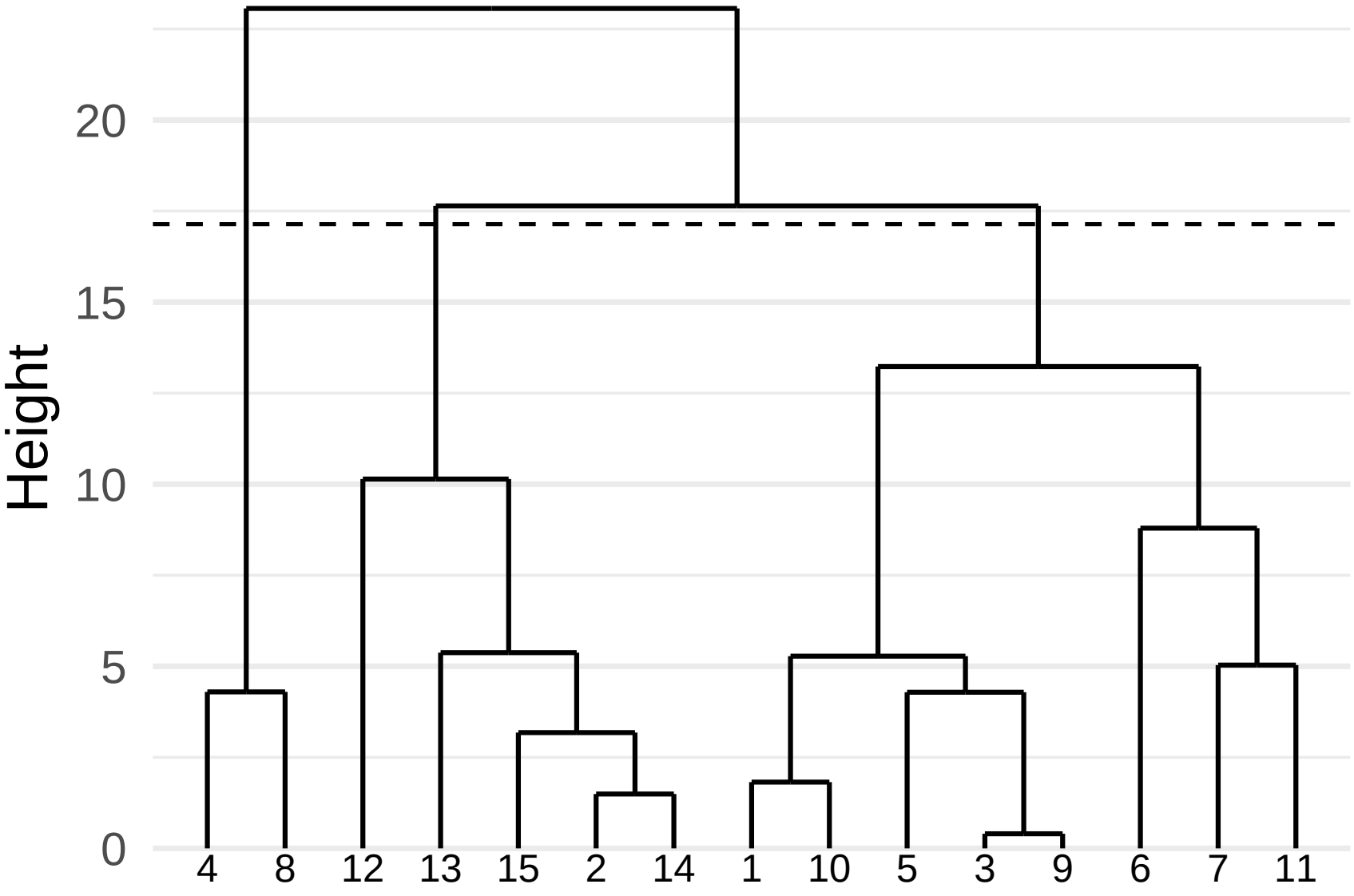
Example dendrogram



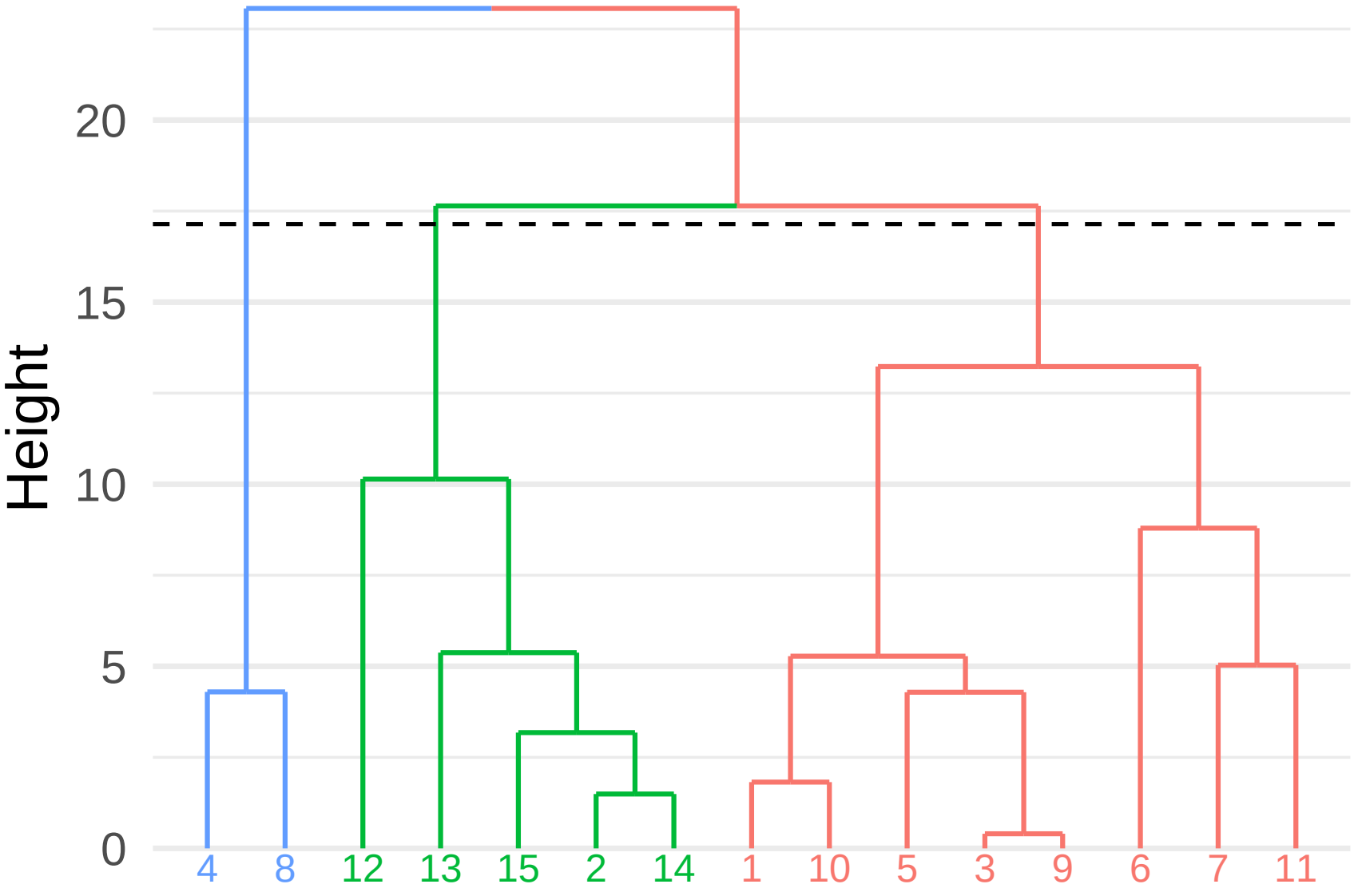
Example dendrogram



Example dendrogram



Example dendrogram



Pros and Cons of Hierarchical Clustering

Advantages

- Deterministic, unlike K-means
- No need to decide on K in advance (although can specify as a stopping condition)
- Allows hierarchical relations to be examined (usually through *dendrograms*)

Disadvantages

- More complex to compute
- The decision about where to create branches and in what order can be somewhat arbitrary, determined by method of declaring the "distance" to already formed clusters

Learning from, and interpreting, clusters

Point of clustering: to *discover* new ways to classify a set of observations.

This means:

1. Clusters are *dataset-dependent* — interpretation of clusters depends on sample of observations (and features)

2. The way you *label* clusters is crucial, and potentially misleading

Labelling clusters

In my view, one of the most controversial parts of clustering is **labelling**.

Analyst takes a statistically-created contraption and attempts to make it meaningful with concise labels.

This is necessary: the consumer/reader needs an interpretation.

But it can also be sketchy — people disagree on interpretations.

Unfortunately: that's life. But we can try to be more principled...

Guiding principles

1. Choose number of clusters *carefully*, *thoughtfully* and *transparently* as this will affect how you interpret the resulting clusters.
2. Carefully examine a random selection of observations in each cluster you estimate (between 10 and 30); take notes and synthesise notes into a corresponding label that matches observations in cluster.
3. Have multiple people provide labels and compare; run experiments to see if subjects broadly agree with your interpretations.
4. Provide list of most "distinctive" features of each cluster (if possible).
5. Check if your cluster labels/interpretations hold up after using different clustering approaches.
6. Always provide all the details of your process in **replication materials**.